

Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Kierunek: Informatyka Stosowana (IST)

Specjalność: Zastosowania Specjalistycznych Technologii Informatycznych (ZSTI)

PRACA DYPLOMOWA
MAGISTERSKA

Wykrywanie Fake News przy użyciu metod
zespołowego uczenia maszynowego

Vadym Liss

Opiekun pracy
Dr inż. Bernadetta Maleszka

Słowa kluczowe: uczenie maszynowe, Fake News, sztuczna inteligencja

WROCŁAW 2025

STRESZCZENIE

Od wiosny 2022 roku uczenie maszynowe zyskało ogromną popularność jako narzędzie do rozwiązywania szerokiego zakresu problemów. Zarówno duże korporacje technologiczne, jak i małe zespoły oraz indywidualni użytkownicy dążą do integracji sztucznej inteligencji w swoich produktach i aplikacjach. Niezależnie od poziomu wiedzy technicznej, coraz więcej osób dostrzega rosnącą rolę SI w niemal każdej dziedzinie. Chociaż rozwój ten niesie wiele korzyści, wiąże się również z poważnymi zagrożeniami. Rozpowszechnienie sztucznej inteligencji ułatwiło tworzenie i dystrybucję treści dezinformacyjnych, co zagraża obiektywizmowi i może prowadzić do manipulacji informacjami. W jednym z najnowszych badań [4] autorzy zwracają uwagę na narastający problem fałszywych wiadomości w mediach społecznościowych. Pomimo postępów w dziedzinie sztucznej inteligencji, wykrywanie i przeciwdziałanie dezinformacji pozostaje znaczącym wyzwaniem. Niniejszy przegląd analizuje obecne metody oraz przyszłe kierunki rozwoju w walce z tym kluczowym problemem. W ramach tej pracy przeprowadzono przegląd istniejących metod i narzędzi służących do ograniczania treści dezinformacyjnych, ze szczególnym uwzględnieniem ich praktycznego zastosowania w XXI wieku. Zaproponowane rozwiązania opierają się na odpowiednim przetwarzaniu i modyfikacji danych wejściowych, co umożliwi lepszą klasyfikację treści. Praca ta obejmuje przegląd wybranych metod zespołowego uczenia maszynowego do wykrywania i klasyfikacji fałszywych wiadomości, ich implementację, przeprowadzenie testów oraz propozycję nowych rozwiązań. Ostatecznym wynikiem jest analiza porównawcza, uwzględniająca specyfikę różnych źródeł danych i zastosowanych metod.

ABSTRACT

Since the spring of 2022, machine learning has gained tremendous popularity as a tool for addressing a wide range of problems. Both large technology corporations and small teams or individuals are increasingly striving to integrate artificial intelligence into their products and applications. Regardless of their technical expertise, more and more people recognize the rapidly growing role of AI in nearly every domain. While this development brings numerous benefits, it also poses significant challenges. The widespread adoption of AI has facilitated the production and dissemination of disinformation, threatening objecti-

vity and potentially leading to information manipulation. In a recent study by Aimeur et al. (2023), the authors highlight the escalating issue of fake news on social media platforms. Despite progress in artificial intelligence, detecting and countering disinformation remains a formidable challenge. This review examines current methods and explores future directions for tackling this critical problem. This work provides a comprehensive overview of existing methods and tools designed to mitigate disinformation, emphasizing their practical applications in the 21st century. The proposed solutions rely on effective processing and modification of input data to improve content classification. In this paper, we review selected ensemble machine learning methods for detecting and classifying fake news, implement these techniques, conduct tests, and propose new solutions. The final result is a comparative analysis that accounts for the specificities of various data sources and the methods applied.

SPIS TREŚCI

1. Wstęp	4
2. Przegląd literatury	6
2.1. Korzyści zastosowania metod zespołowego uczenia maszynowego	6
2.2. Wprowadzenie do metod zespołowych	7
2.2.1. Wyjaśnienie idei za tymi technikami	7
2.2.2. Dlaczego są skuteczne w wykrywaniu fake newsów?	8
2.2.3. Ważne pojęcia	9
2.3. Kluczowe modele w analizie i detekcji dezinformacji	10
2.3.1. Regresja logistyczna	10
2.3.2. Random Forest	10
2.3.3. Boosting: AdaBoost i Gradient Boosting	11
2.3.4. XGBoost i LightGBM	11
2.3.5. Stacking	12
2.3.6. Sieci neuronowe	12
2.3.7. Voting Classifier	12
2.3.8. Modele z post-processingiem	13
2.3.9. CatBoost	13
2.4. Reprezentacje semantyczne tekstu	13
2.4.1. BERT (Bidirectional Encoder Representations from Transformers)	13
2.4.2. RoBERTa (Robustly Optimized BERT Approach)	14
2.4.3. Sentence-BERT (SBERT)	14
2.4.4. Porównanie wspomnianych reprezentacji	15
2.5. Techniki uczenia w różnych kontekstach danych	15
2.5.1. One-Shot Learning (OSL)	15
2.5.2. Few-Shot Learning (FSL)	15
2.5.3. Porównanie technik uczenia w różnych kontekstach danych	16
2.6. Implementacja przygotowania danych	16
2.7. Implementacja wybranych naukowych metod	17
2.7.1. Modele oparte na drzewach	18
2.7.2. Sieci neuronowe	19
2.7.3. Zespoły klasyfikatorów	21
2.7.4. Modele zoptymalizowane	33
2.8. Opis zestawów danych użytych w eksperymentach	34
2.8.1. ISOT Fake News Dataset	34

2.8.2.	WELFake Dataset	36
2.8.3.	BuzzFeed News Dataset	37
2.8.4.	Porównanie zbiorów danych	38
2.9.	Implementacja analizy wyników	39
2.10.	Wstępna analiza wyników	41
2.10.1.	Dokładność (Accuracy)	42
2.10.2.	Precyzja (Precision)	42
2.10.3.	Recall	43
2.10.4.	Wynik F1 (F1-Score)	44
2.10.5.	Logarytmiczna strata (Log-Loss)	45
2.10.6.	Współczynnik korelacji Matthews (MCC)	45
2.10.7.	Pole pod krzywą ROC-AUC (ROC-AUC)	46
2.10.8.	Współczynnik Kappa Cohena (Cohen's Kappa)	47
2.10.9.	Średnia (Mean)	47
2.10.10.	Czas wykonania (Execution Time)	48
2.10.11.	Podsumowanie wstępnej analizy wyników metod	49
3.	Własne metody	51
3.1.	Pierwsza metoda (Metoda17)	51
3.1.1.	Pomysł na metodę	51
3.1.2.	Wejściowe parametry	51
3.1.3.	Wyjściowe parametry	52
3.1.4.	Algorytm	52
3.1.5.	Kod	52
3.1.6.	Właściwości analityczne	54
3.2.	Druża metoda (Metoda18)	54
3.2.1.	Pomysł na metodę	54
3.2.2.	Wejściowe parametry	55
3.2.3.	Wyjściowe parametry	55
3.2.4.	Algorytm	55
3.2.5.	Kod	56
3.2.6.	Właściwości analityczne	57
3.3.	Trzecia metoda (Metoda19)	57
3.3.1.	Pomysł na metodę	57
3.3.2.	Wejściowe dane	57
3.3.3.	Wyjściowe dane	58
3.3.4.	Algorytm	58
3.3.5.	Kod	59
3.3.6.	Właściwości analityczne	61
3.4.	Czwarta metoda (Metoda20)	61
3.4.1.	Pomysł na metodę	61
3.4.2.	Wejściowe dane	62

3.4.3.	Wyjściowe dane	62
3.4.4.	Algorytm	62
3.4.5.	Kod	63
3.4.6.	Właściwości analityczne	64
4.	Szczegóły implementacyjne	65
4.1.	Technologie	65
4.2.	Wymagania systemowe	66
4.2.1.	Sprzęt	66
4.2.2.	Oprogramowanie	66
4.3.	Architektura Systemu	67
4.3.1.	Warstwa Przygotowania Danych	68
4.3.2.	Warstwa Trenowania Modeli	68
4.3.3.	Warstwa Oceny Wyników	69
4.3.4.	Warstwa Wizualizacji	69
	Bibliografia	70
	Spis rysunków	72
	Dodatki	73

1. WSTĘP

Tematem niniejszej pracy magisterskiej jest *Wykrywanie Fake News przy użyciu metod zespołowego uczenia maszynowego*. Problem dezinformacji, w tym tak zwanego fake news, jest jednym z najpoważniejszych wyzwań współczesnego społeczeństwa informacyjnego.

Motywacją do podjęcia tego tematu wynika z realnych zagrożeń, jakie niesie za sobą dezinformacja oraz potrzeby opracowania skutecznych narzędzi do jej zwalczania. W ostatnich latach zaobserwowano liczne przypadki wpływu **fake news** na wyniki wyborów, podziały społeczne czy reakcje na kryzysy zdrowotne, takie jak pandemia COVID-19. Informacje nieprawdziwe, ale celowo rozpowszechniane, potrafią wywoływać panikę, dezinformować w kluczowych sprawach publicznych, a nawet prowadzić do eskalacji konfliktów międzynarodowych. W tym kontekście rola zaawansowanych technologii informatycznych w przeciwdziałaniu tym zjawiskom staje się kluczowa.

Dodatkową motywacją do podjęcia tematu jest rosnące zainteresowanie zastosowaniem sztucznej inteligencji w analizie danych tekstowych oraz ciągła potrzeba udoskonalania istniejących rozwiązań. Współczesne systemy detekcji **fake news** często bazują na standardowych algorytmach, które, mimo wysokiej popularności, nie zawsze radzą sobie w sytuacjach wymagających analizy danych o dużej zmienności, nierównowadze klas czy subtelnych różnicach semantycznych. Opracowanie nowych metod, które potrafią efektywnie radzić sobie z tymi wyzwaniami, może znacząco przyczynić się do zwiększenia skuteczności detekcji **fake news**.

W odpowiedzi na te wyzwania, dziedzina sztucznej inteligencji, a w szczególności uczenie maszynowe, oferuje zaawansowane narzędzia umożliwiające skuteczne wykrywanie i klasyfikację dezinformacji. Wśród tych narzędzi szczególną rolę odgrywają metody zespołowego **uczenia maszynowego** (*ensemble learning*), które dzięki integracji wyników wielu modeli bazowych, pozwalają na osiąganie wyższej dokładności predykcji i większej stabilności wyników. Techniki te, takie jak **bagging**, **boosting** czy **stacking**, zdobyły uznanie dzięki swojej skuteczności w rozwiązywaniu problemów nierównowagi klas oraz w zadaniach wymagających analizy dużych i zróżnicowanych zbiorów danych.

Celem niniejszej pracy jest opracowanie i implementacja autorskich metod zespołowego uczenia maszynowego dedykowanych do detekcji **fake news** oraz analiza ich skuteczności w porównaniu z tradycyjnymi podejściami. Praca koncentruje się na tworzeniu innowacyjnych rozwiązań, które wykraczają poza standardowe techniki. Szczególny nacisk położono na projektowanie algorytmów i użycie modeli które łączą w sobie no-

woczesne podejścia do przetwarzania języka naturalnego z zaawansowanymi technikami optymalizacji wyników klasyfikacji.

Główne założenia pracy obejmują:

1. Opracowanie nowych metod zespołowych wykorzystujących.
2. Analizę efektywności autorskich rozwiązań w kontekście kluczowych miar, takich jak **Log Loss**, **Accuracy** i **Execution Time**.
3. Praktyczne zastosowanie proponowanych metod na różnych zbiorach danych zawierających fałszywe i prawdziwe wiadomości, w tym na benchmarkowych zestawach, takich jak ISOT Fake News Dataset i innych.

Celem pracy jest również zbadanie, które podejścia w ramach metod zespołowych okazują się najskuteczniejsze w detekcji dezinformacji oraz jak innowacyjne rozwiązania mogą przyczynić się do dalszego zwiększenia efektywności tych systemów. Dzięki temu niniejsze badania mają na celu nie tylko teoretyczne rozwinięcie wiedzy w obszarze wykrywania **fake news**, ale również dostarczenie praktycznych narzędzi, które mogą być wykorzystywane zarówno w badaniach naukowych, jak i w zastosowaniach przemysłowych.

Wykrywanie fake news to złożone zadanie, które wymaga nie tylko zaawansowanych algorytmów, ale także odpowiedniego przetwarzania danych wejściowych. Przedstawione w tej pracy podejścia bazują się na połączeniu między tradycyjnymi technikami uczenia maszynowego a nowoczesnymi metodami przetwarzania języka naturalnego. W efekcie celem pracy jest stworzenie kompleksowego obrazu możliwości i ograniczeń współczesnych metod zespołowych w wykrywaniu **fake news**, a także wskazanie dalszych kierunków rozwoju w tym obszarze.

Niniejsze badania mają na celu nie tylko teoretyczne rozwinięcie wiedzy w obszarze wykrywania fake news, ale również dostarczenie praktycznych narzędzi, które mogą być wykorzystywane zarówno w badaniach naukowych, jak i w zastosowaniach komercyjnych.

Praca ta stanowi próbę odpowiedzi na pytanie, które podejścia w ramach metod zespołowych okazują się najskuteczniejsze w detekcji dezinformacji, oraz jakie innowacyjne rozwiązania mogą jeszcze bardziej zwiększyć efektywność takich systemów. Dzięki temu niniejsze badania przyczynią się do rozwinięcia narzędzi, które mogą być wykorzystywane zarówno w badaniach naukowych, jak i w praktycznych zastosowaniach w walce z dezinformacją.

2. PRZEGLĄD LITERATURY

2.1. KORZYŚCI ZASTOSOWANIA METOD ZESPOŁOWEGO UCZENIA MASZYNOWEGO

Jak wspomniano w podanym artykule [8], **ensemble learning** to metoda uczenia maszynowego, która polega na łączeniu wyników wielu modeli w celu poprawy dokładności predykcji. Główne klasy metod ensemble learning obejmują **bagging**, **boosting**, **stacking** oraz inne. Każda z tych technik znajduje zastosowanie w rozwiązaniu problemów nierównowagi klas (class imbalance), gdzie liczba próbek jednej klasy jest znacznie większa od liczby próbek drugiej klasy. Takie sytuacje mogą prowadzić do stronniczości modeli maszynowych na korzyść klasy większościowej, co prowadzi do błędnej klasyfikacji próbek klasy mniejszościowej. Bardziej szczegółowy opis wspomnianych powyżej metod zostanie omówiony w następnym podrozdziale.

W artykule wykazano, że metody ensemble są szczególnie skuteczne w przypadku danych nierównoważnych, takich jak:

- Wykrywanie oszustw (fraud detection),
- Analiza sentymentu,
- Diagnostyka medyczna,
- Prognozowanie awarii urządzeń.

W porównaniu do pojedynczych modeli, metody zespołowe osiągają lepsze wyniki zarówno w metrykach klasyfikacyjnych, jak i ogólnej dokładności. Ponadto, odpowiednio zaprojektowane metody ensemble mogą redukować problem nadmiernego dopasowania (overfitting). Chcielibyśmy wyróżnić następujące **zalety** stosowania metod zespołowego uczenia maszynowego:

- **Zwiększona odporność na błędy:** Ensemble learning zmniejsza ryzyko błędów spowodowanych nadmierną wariancją lub stronniczością pojedynczych modeli, co zapewnia bardziej stabilne i wiarygodne wyniki.
- **Integracja różnorodnych modeli:** Możliwość łączenia różnych typów modeli pozwala na wykorzystanie ich indywidualnych mocnych stron, co zwiększa skuteczność w zadaniach klasyfikacyjnych i regresyjnych.
- **Skuteczność w rozwiązywaniu problemów nierównowagi klas:** Ensemble learning wykazuje lepsze wyniki w zadaniach z nierównomiernym rozkładem klas, zwłaszcza

gdzie jest stosowany w połączeniu z metodami wzbogacania danych, takimi jak SMOTE czy generatywne sieci przeciwstawne (GANs).

2.2. WPROWADZENIE DO METOD ZESPOŁOWYCH

2.2.1. Wyjaśnienie idei za tymi technikami

Rozszerzając wiedzę na temat technik metod zespołowego uczenia maszynowego, szeroko opisanych w artykule [5], można zauważyć ich uniwersalność i znaczenie w różnych zastosowaniach. Jak wskazano w artykule, sukces tych metod wynika z ich zdolności do redukowania błędów systematycznych (bias) i wariancji, efektywnego wykorzystania zasobów obliczeniowych oraz umiejętności reprezentowania bardziej złożonych zależności w danych. W praktyce metody ensemble znajdują zastosowanie w wielu dziedzinach, takich jak diagnostyka medyczna, analiza obrazów, wykrywanie anomalii czy przewidywanie awarii urządzeń. Autorzy podkreślają również ich skuteczność w rozwiązywaniu problemów nierównowagi klas, co jest szczególnie istotne w zadaniach, gdzie jedna klasa danych jest znacząco mniej reprezentowana.

Dzięki integracji z modelami głębokiego uczenia, metody zespołowe łączą zdolność do hierarchicznego uczenia reprezentacji z większą precyzją predykcji. Jednocześnie rozwój tych modeli wymaga rozwiązywania wyzwań, takich jak wysoki koszt obliczeniowy oraz konieczność zapewnienia różnorodności pomiędzy modelami bazowymi, co pozwala na ich skuteczne wykorzystanie w praktyce.

Bagging (Bootstrap Aggregating)

Bagging, znany także jako bootstrap aggregating, to metoda redukująca wariancję modeli bazowych poprzez trenowanie ich na różnych, losowych próbkach danych wybranych z zamianą (bootstrap). Jak wspomniano w artykule, bagging polega na tworzeniu wielu niezależnych modeli bazowych, których wyniki są następnie łączone poprzez głosowanie większościowe (w przypadku klasyfikacji) lub uśrednianie (w przypadku regresji).

Przykładem zastosowania baggingu jest **Random Forest**, w którym dodatkowo losowo wybierane są podzbiory cech na każdym etapie budowania drzewa decyzyjnego. W ten sposób zwiększa się różnorodność modeli bazowych, zmniejszając jednocześnie ryzyko nadmiernego dopasowania i poprawiając stabilność wyników.

Boosting

Boosting to metoda polegająca na iteracyjnym trenowaniu modeli, gdzie każdy kolejny model skupia się na przykładach trudnych do sklasyfikowania przez poprzedników. W artykule podkreślono, że proces ten umożliwia tworzenie silnych klasyfikatorów zdolnych do radzenia sobie z bardziej złożonymi problemami.

Przykładem jest **AdaBoost**, który nadaje większe wagi błędnie sklasyfikowanym przykładom w kolejnych iteracjach, minimalizując funkcję błędu eksponencjalnego. Natomiast **Gradient Boosting** rozszerza to podejście, optymalizując dowolną różniczkowalną funkcję błędu. Boosting efektywnie redukuje błąd systematyczny (bias) i wariancję, co prowadzi do lepszych wyników.

Stacking (Stacked Generalization)

Stacking, czyli uogólnione zestawianie (stacked generalization), to metoda zespołowa, w której różne modele bazowe są łączone za pomocą meta-modelu. Meta-model uczy się, jak najlepiej łączyć prognozy modeli bazowych, korzystając z ich wyników jako danych wejściowych.

Jak podano w artykule, stacking pozwala na łączenie różnych typów modeli (drzewa decyzyjne, sieci neuronowe, SVM), co czyni go bardziej elastycznym i zdolnym do generalizacji. W literaturze metoda ta jest również stosowana w złożonych problemach, takich jak klasyfikacja obrazów czy przewidywanie trendów.

Negative Correlation Learning (NCL)

W technice **Negative Correlation Learning (NCL)** dąży się do zwiększenia różnorodności wśród modeli bazowych poprzez wprowadzenie funkcji kosztu, która karze zbyt podobne prognozy między modelami. W artykule podkreślono, że metoda ta jest szczególnie skuteczna w problemach wymagających wysokiej zdolności do generalizacji.

Przykładem zastosowania NCL są zadania regresyjne i klasyfikacyjne, w których różnorodność modeli poprawia ogólną wydajność zespołu.

Homogeniczne i heterogeniczne zespoły

- **Homogeniczne zespoły:** Wykorzystują jeden typ klasyfikatorów (drzewa decyzyjne w Random Forest). Kluczowym wyzwaniem jest wprowadzenie różnorodności, poprzez losowe próbkowanie danych treningowych lub cech.
- **Heterogeniczne zespoły:** Łączą różne typy modeli (sieci neuronowe, SVM, regresję logistyczną). Są bardziej elastyczne i zdolne do uchwycenia różnych cech danych, co czyni je użytecznymi w złożonych problemach, takich jak analiza tekstu czy przewidywanie awarii.

2.2.2. Dlaczego są skuteczne w wykrywaniu fake newsów?

Bagging (Bootstrap Aggregating)

Bagging zwiększa stabilność modeli w sytuacjach, gdy dane są niestabilne lub różnorodne – co jest powszechne w zadaniach wykrywania fake news. Przykładem jest Random Forest, który dzięki losowemu wyborowi podzbiorów cech i danych jest odporny

na przetrenowanie, co pozwala mu skutecznie radzić sobie z różnymi rodzajami tekstów. Redukcja wariancji umożliwia bardziej spójne klasyfikowanie nawet w przypadku danych zawierających subtelne różnice.

Boosting

Boosting koncentruje się na trudno klasyfikowalnych przykładach, które są częste w zbiorach danych dotyczących fake news. Fałszywe informacje często charakteryzują się subtelnymi manipulacjami w języku, co wymaga modeli zdolnych do uchwycenia takich szczegółów. Boosting, w formie Gradient Boostingu lub XGBoost, jest w stanie modelować złożone zależności między cechami, co prowadzi do wyższej dokładności w identyfikowaniu fałszywych wiadomości.

Stacking (Stacked Generalization)

Stacking umożliwia łączenie różnych metod analizy tekstu, takich jak modele oparte na n-gramach, TF-IDF, czy głębokie sieci neuronowe. Dzięki temu można wykorzystać mocne strony każdego z tych podejść, jednocześnie minimalizując ich słabości. W zadaniach wykrywania fake news stacking pozwala połączyć modele bazujące na różnych źródłach danych (tekst, metadane, analiza kontekstu), co daje bardziej kompleksowy obraz problemu.

Negative Correlation Learning (NCL)

NCL zwiększa różnorodność modeli bazowych, co sprawdza się w wykrywaniu fake news, gdzie dane są niejednorodne i zawierają różnorodne wzorce. Dzięki promowaniu różnorodności w predykcjach, modele uczą się szerokiego zakresu cech, co prowadzi do lepszego uogólnienia wyników na danych testowych.

Homogeniczne i heterogeniczne zespoły

Homogeniczne zespoły, dobrze radzą sobie z dużą liczbą cech tekstowych i ich interakcjami. Z kolei heterogeniczne zespoły, łączące różne typy modeli, pozwalają lepiej uchwycić różnorodne cechy danych fake news, takie jak manipulacyjny język, brak źródeł czy określone wzorce emocjonalne w tekstach.

2.2.3. Ważne pojęcia

Binarną entropia krzyżowa (Binary Cross-Entropy) to sposób mierzenia, jak dobrze coś jest klasyfikowane na dwie grupy, na przykład czy wiadomość jest prawdziwa, czy fałszywa. W praktyce pomaga sprawdzić, czy nasze przewidywania są bliskie rzeczywistości - im mniejszy błąd, tym lepiej rozróżniamy te dwie kategorie, co jest kluczowe w analizie wiarygodności treści.

Metoda wczesnego zatrzymania (Early Stopping Rounds) to technika, która pozwala zatrzymać proces uczenia, gdy dalsze próby nie przynoszą poprawy. Wyobraźmy sobie, że

uczymy się rozpoznawać fałszywe wiadomości: jeśli po kilku rundach nie robimy postępów, przerywamy, aby nie tracić czasu i nie skupiać się za bardzo na szczegółach, które mogą być mylące w nowych sytuacjach.

Losowy wybór (Random State) zapewnia, że losowe wybory, na przykład podział danych na grupy do analizy, są zawsze takie same. Dzięki temu wyniki badań są powtarzalne - jeśli analizujemy wiadomości, możemy być pewni, że za każdym razem pracujemy na tych samych zestawach, co ułatwia porównanie i zapewnia spójność.

2.3. KLUCZOWE MODELE W ANALIZIE I DETEKCJI DEZINFORMACJI

2.3.1. Regresja logistyczna

Regresja logistyczna jest modelem statystycznym stosowanym głównie w klasyfikacji binarnej. Opiera się na wykorzystaniu funkcji sigmoidalnej, która przekształca wyniki liniowej kombinacji cech wejściowych w wartość prawdopodobieństwa, umożliwiając klasyfikację na dwie klasy. Model ten jest szczególnie ceniony za prostotę i interpretowalność - współczynniki regresji wskazują na wpływ poszczególnych cech na wynik predykcji.

W praktyce regresja logistyczna jest wykorzystywana w analizach, takich jak:

- **Prognozowanie ryzyka chorób** w medycynie,
- **Segmentacja klientów** w marketingu,
- **Ocena ryzyka kredytowego** w finansach.

Największym ograniczeniem regresji logistycznej jest jej niezdolność do modelowania złożonych, nieliniowych zależności oraz tendencja do faworyzowania klasy dominującej w przypadku nierównoważnych danych.

2.3.2. Random Forest

Random Forest to rozwinięcie drzew decyzyjnych, które wprowadza element losowości, aby zwiększyć różnorodność modeli bazowych i zmniejszyć ich podatność na przeuczenie. Model ten wykorzystuje:

- **Bagging** (Bootstrap Aggregating): losowe próbkowanie z zamianą, aby stworzyć wiele zbiorów danych treningowych,
- **Losowy wybór cech** na każdym etapie budowy drzewa, co zmniejsza korelację między drzewami.

Według przeprowadzonej analizy zastosowań, Random Forest jest przydatny w zadaniach takich jak:

- **Diagnoza medyczna**, gdzie umożliwia ocenę ważności cech diagnostycznych,
- **Wykrywanie intruzji** w systemach bezpieczeństwa,

- **Analiza danych obrazowych**, dzięki możliwości przetwarzania dużych zbiorów w sposób równoległy.

Pomimo swoich zalet, Random Forest jest trudniejszy w interpretacji w porównaniu do pojedynczych drzew decyzyjnych.

2.3.3. Boosting: AdaBoost i Gradient Boosting

Boosting to rodzina algorytmów, które iteracyjnie wzmacniają słabe klasyfikatory. Każda iteracja skupia się na poprawie błędów poprzednich, co prowadzi do bardziej precyzyjnego modelu.

- **AdaBoost**: Używa prostych modeli i nadaje większe wagi przykładom błędnie sklasyfikowanym. Znajduje zastosowanie w:
 - **Wykrywaniu intruzji** w systemach komputerowych,
 - **Analizie tekstu**,
 - **Prognozowaniu upadłości firm**.
- **Gradient Boosting**: Wykorzystuje optymalizację gradientową, aby iteracyjnie korygować błędy. Modeluje złożone zależności nieliniowe, co sprawia, że jest bardzo dokładny, ale wymaga starannego strojenia hiperparametrów. Zastosowania obejmują:
 - **Diagnozę medyczną**,
 - **Prognozowanie sprzedaży**,
 - **Analizę ryzyka kredytowego**.

Boosting jest bardziej podatny na szum w danych, ale pozwala uzyskać wysoką dokładność nawet w trudnych zadaniach.

2.3.4. XGBoost i LightGBM

Te zaawansowane modele boostingu zostały zoptymalizowane pod kątem wydajności i skalowalności:

- **XGBoost**: Wprowadza regularyzację (L1 i L2) i przetwarzanie równoległe, co pozwala na szybkie trenowanie dużych zbiorów danych. Jest szeroko stosowany w:
 - **Prognozowaniu cen akcji**,
 - **Wykrywaniu rzadkich chorób**,
 - **Analizie danych przestrzennych**.
- **LightGBM**: Korzysta z technik takich jak budowa drzewa na podstawie histogramów, co znacząco przyspiesza trenowanie. Zastosowania obejmują:
 - **Wykrywanie oszustw**,
 - **Analizę tekstu, obrazów i dźwięków**.

Oba modele oferują wysoką dokładność, ale ich strojenie może być bardziej skomplikowane w porównaniu do prostszych rozwiązań.

2.3.5. Stacking

Stacking to metoda łączenia różnych modeli bazowych za pomocą meta-modelu. Meta-model uczy się, jak najlepiej łączyć prognozy, co pozwala na:

- **Uwzględnienie różnorodnych aspektów danych,**
- **Zwiększenie skuteczności poprzez integrację różnych podejść.**

Stacking jest szczególnie efektywny w zadaniach, gdzie dane mają skomplikowaną strukturę, a różne modele bazowe mogą uchwycić różne cechy. Zastosowania obejmują:

- **Analiza sentymentu w mediach społecznościowych,**
- **Rozpoznawanie mowy.**

2.3.6. Sieci neuronowe

Sieci neuronowe to zaawansowane modele klasyfikacyjne oparte na warstwach neuronów, które przetwarzają dane wejściowe w sposób inspirowany ludzkim mózgiem. Pozwalają na:

- **Modelowanie złożonych, nieliniowych zależności między cechami,**
- **Przetwarzanie różnorodnych typów danych, takich jak tekst czy sekwencje.**

Sieci neuronowe są szczególnie efektywne w zadaniach, gdzie dane mają strukturę sekwencyjną lub przestrzenną. W kontekście klasyfikacji obiekt (wektor cech lub tekst) przechodzi przez moduł klasyfikacji składający się z warstw neuronowych, a wynikiem jest etykieta binarna określająca grupę przynależności. Zastosowania obejmują:

- **Wykrywanie dezinformacji w tekstach mediów społecznościowych,**
- **Rozpoznawanie obrazów i analiza danych medycznych.**

2.3.7. Voting Classifier

Voting Classifier to technika zespołowa, która integruje predykcje wielu klasyfikatorów za pomocą mechanizmu głosowania. Umożliwia:

- **Zwiększenie stabilności predykcji poprzez konsensus,**
- **Wykorzystanie różnorodnych podejść klasyfikacyjnych.**

Metoda ta jest skuteczna w zadaniach wymagających niezawodności i odporności na błędy pojedynczych modeli. Obiekt do klasyfikacji (wektor cech) jest analizowany przez moduł złożony z równoległe działających klasyfikatorów, a wynikiem jest etykieta binarna wyłoniona w procesie głosowania. Zastosowania obejmują:

- **Analiza sentymentu w danych tekstowych,**
- **Wykrywanie oszustw w systemach finansowych.**

2.3.8. Modele z post-processingiem

Modele z post-processingiem to podejście, które wzbogaca klasyfikację o dodatkowe kroki korygujące oparte na heurystykach lub analizie niepewności. Pozwalają na:

- **Poprawę precyzji predykcji dzięki dodatkowym informacjom kontekstowym,**
- **Uwzględnienie niepewności modelu w procesie decyzyjnym.**

Są one szczególnie przydatne w specyficznych zadaniach, gdzie standardowa klasyfikacja wymaga doprecyzowania. Obiekt do analizy, tekst lub wektor cech, przechodzi przez moduł klasyfikacji, a następnie jego etykieta binarna jest korygowana na podstawie meta-danych, dając ostateczną grupę przynależności. Zastosowania obejmują:

- **Wykrywanie dezinformacji z uwzględnieniem wiarygodności źródła,**
- **Analiza ryzyka w danych finansowych.**

2.3.9. CatBoost

CatBoost to algorytm boostingu zoptymalizowany do pracy z danymi kategorycznymi, który iteracyjnie ulepsza klasyfikację. Umożliwia:

- **Efektywne przetwarzanie zmiennych kategorycznych bez dodatkowego kodowania,**
- **Wysoką dokładność dzięki gradientowemu podejściu do optymalizacji.**

CatBoost sprawdza się w zadaniach z heterogenicznymi danymi. Obiekt do klasyfikacji, wektor cech z danymi kategorycznymi, jest analizowany przez moduł oparty na drzewach decyzyjnych, a wynikiem jest etykieta binarna określająca grupę. Zastosowania obejmują:

- **Ocena ryzyka kredytowego w finansach,**
- **Prognozowanie sprzedaży w e-commerce.**

2.4. REPREZENTACJE SEMANTYCZNE TEKSTU

W ostatnich latach modele głębokiego uczenia, w szczególności te oparte na architekturze transformatorowej, zrewolucjonizowały przetwarzanie języka naturalnego (NLP). W kontekście generowania osadzeń tekstowych, technologie te umożliwiły znaczący postęp w jakości reprezentacji semantycznych zdań. Poniżej przedstawiono szczegółowy opis trzech kluczowych metod omawianych w artykule [16], z uwzględnieniem ich zalet, ograniczeń oraz zastosowań.

2.4.1. BERT (Bidirectional Encoder Representations from Transformers)

Model BERT stanowi przełom w NLP dzięki dwukierunkowemu przetwarzaniu tekstu, co umożliwia pełne uchwycenie kontekstu zdania. Dzięki warstwom transformatorowym BERT ustanowił nowe standardy w takich zadaniach jak klasyfikacja zdań czy analiza

podobieństwa tekstowego (STS). Jego zastosowanie wymaga jednak jednoczesnego podania obu zdań do sieci, co prowadzi do ogromnych kosztów obliczeniowych. Przykładowo, analiza 10 000 zdań za pomocą BERT wymaga przeprowadzenia około 50 milionów obliczeń (~65 godzin na nowoczesnej GPU).

Pomimo sukcesów, BERT nie generuje osadzeń zdań w sposób natywny. Stosowane podejścia, takie jak uśrednianie wyjść z warstwy końcowej (average pooling) czy wykorzystanie tokenu (CLS), często prowadzą do wyników gorszych od metod tradycyjnych, takich jak GloVe. Zastosowanie BERT w zadaniach takich jak klasteryzacja czy wyszukiwanie semantyczne jest ograniczone ze względu na niską jakość generowanych osadzeń oraz bardzo wysokie wymagania obliczeniowe.

2.4.2. RoBERTa (Robustly Optimized BERT Approach)

RoBERTa to ulepszona wersja BERT, zoptymalizowana pod kątem wydajności w zadaniach z nadzorem. Dzięki dłuższemu treningowi, większej ilości danych i lepszej optymalizacji hiperparametrów, RoBERTa osiąga lepsze wyniki w zadaniach klasyfikacyjnych. Usunięcie tokenów (SEP) i (CLS) w procesie pretreningu pozwoliło na zwiększenie efektywności modelu.

Jednak w kontekście generowania osadzeń zdań RoBERTa wykazuje wyniki porównywalne z BERT. W implementacji SBERT (jako SRoBERTa) nie przyniosła istotnej przewagi nad standardowym SBERT. RoBERTa pozostaje bardziej wydajna w zadaniach klasyfikacyjnych, lecz jej zastosowanie w klasteryzacji czy wyszukiwaniu semantycznym jest ograniczone podobnie jak w przypadku BERT.

2.4.3. Sentence-BERT (SBERT)

SBERT stanowi odpowiedź na ograniczenia BERT w generowaniu osadzeń zdań. Wykorzystuje sieci syjamskie (Siamese Networks), które umożliwiają jednoczesne przetwarzanie dwóch zdań za pomocą identycznych wag modelu BERT. Wynikowe osadzenia są porównywane przy użyciu miar, takich jak kosinusowe podobieństwo, co pozwala na szybkie określenie relacji semantycznych między zdaniem.

Jednym z kluczowych usprawnień SBERT jest wprowadzenie warstwy "pooling", która uśrednia wartości tokenów (MEAN), wybiera wartości maksymalne (MAX) lub wykorzystuje token (CLS). Model ten został przeszkolony na zbiorach NLI (Natural Language Inference) oraz STS. Dzięki temu SBERT znacząco redukuje czas obliczeń – analiza 10 000 zdań zajmuje wielokrotnie szybciej w porównaniu do BERT.

SBERT przewyższa jakością osadzenia generowane przez BERT, Universal Sentence Encoder czy InferSent. Na przykład w zadaniach STS osiągnął średnią poprawę o 11.7 punktów w porównaniu do InferSent i 5.5 punktów w stosunku do Universal Sentence Encoder.

2.4.4. Porównanie wspomnianych reprezentacji

Metoda	Opis	Zalety	Ograniczenia
BERT	Generuje osadzenia tokenów dla klasyfikacji zdań i STS.	Dwukierunkowe przetwarzanie, wszechstronność, nowe standardy NLP.	Nie generuje samodzielnych osadzeń, wysokie koszty obliczeń.
RoBERTa	Ulepszony BERT do zadań nadzorowanych NLP.	Lepsza wydajność w klasyfikacji, usunięcie ograniczeń BERT.	Brak poprawy jakości osadzeń względem BERT, wymagania obliczeniowe.
SBERT	Syjamskie sieci BERT do semantycznych osadzeń zdań.	Wysokiej jakości osadzenia, szybka analiza, wszechstronność.	Wymaga dodatkowego treningu, zależność od jakości danych.

Tabela 2.1: Porównanie metod osadzeń tekstowych.

SBERT to istotny krok naprzód w generowaniu osadzeń zdań, który łączy precyzję i elastyczność modelu BERT z wysoką efektywnością obliczeniową. Dzięki zdolności do szybkiego tworzenia wysokiej jakości osadzeń, SBERT znajduje zastosowanie w zadaniach takich jak klasteryzacja, wyszukiwanie semantyczne oraz analiza podobieństwa tekstowego. Przeanalizujemy jakość osadzeń i zbadajemy, czy faktycznie SBERT zapewni lepsze wyniki, wskazując konkretne zalety i ograniczenia SBERT w kontekście wybranych zastosowań.

2.5. TECHNIKI UCZENIA W RÓŻNYCH KONTEKSTACH DANYCH

One-Shot Learning (OSL) i Few-Shot Learning (FSL) różnią się podejściem do problemu ograniczonych danych treningowych, a wybór odpowiedniej metody zależy od konkretnego przypadku użycia.

2.5.1. One-Shot Learning (OSL)

OSL wyróżnia się błyskawiczną adaptacją do nowych klas, co czyni ją idealnym wyborem w sytuacjach, gdzie czas reakcji ma kluczowe znaczenie, przykładowo przy identyfikacji nowo pojawiających się form fake news. Jest to rozwiązanie szybkie i skuteczne, szczególnie w sytuacjach, gdzie dostępny jest tylko jeden przykład.

2.5.2. Few-Shot Learning (FSL)

FSL, dzięki wykorzystaniu większej liczby przykładów, oferuje większą elastyczność i dokładność. To podejście sprawdzi się lepiej w scenariuszach, gdzie dostęp do kilku przykładów nowych klas jest możliwy, a kluczowa jest stabilność i precyzja modelu.

2.5.3. Porównanie technik uczenia w różnych kontekstach danych

W artykule [13] szczegółowo opisano metodę CAPD, która łączy zalety obu podejść. CAPD umożliwia dynamiczną adaptację modeli w obu przypadkach, dzięki czemu stanowi uniwersalne rozwiązanie dla problemów związanych z ograniczoną dostępnością danych.

Na podstawie analizy przeprowadzonej w artykule można stwierdzić, że **FSL jest bardziej uniwersalne** i lepiej dostosowane do długoterminowych scenariuszy, gdzie jakość klasyfikacji i stabilność modelu mają kluczowe znaczenie. Niemniej jednak, **OSL pozostaje niezastąpione w sytuacjach krytycznych**, gdy czas reakcji ma większe znaczenie niż precyzja.

Dzięki innowacyjnemu podejściu opartemu na CAPD, zarówno OSL, jak i FSL mogą zostać zoptymalizowane pod kątem dynamicznego środowiska, takiego jak wykrywanie fake news, co zostało szczegółowo omówione w artykule. CAPD wykorzystuje zarówno minimalne dane OSL, jak i większe zbiory treningowe FSL, aby zwiększyć zdolności adaptacyjne modelu.

2.6. IMPLEMENTACJA PRZYGOTOWANIA DANYCH

Proces przygotowania danych wejściowych stanowi wspólny etap dla wszystkich implementowanych rozwiązań metod uczenia zespołowego, niezależnie od tego, czy opisujemy implementacje z naukowych artykułów, czy autorskie metody. Proces przygotowania danych został zaimplementowany w skrypcie głównym i obejmuje kilka kluczowych kroków, które zapewniają spójną bazę danych dla dalszego przetwarzania klasyfikacyjnego.

Pierwszym krokiem jest wczytanie danych przy użyciu funkcji `load_and_preprocess_data`. Nasza implementacja wykorzystuje pliki `Fake.csv`, zawierający fałszywe wiadomości, oraz `True.csv`, który obejmuje prawdziwe wiadomości. Funkcja `load_and_preprocess_data` wczytuje oba zbiory, a następnie nadaje im binarne etykiety: 0 dla fałszywych wiadomości i 1 dla prawdziwych, co odpowiada zadaniu klasyfikacji binarnej w wykrywaniu fake newsów. Zwracamy uwagę, że dataset musi zawierać obowiązkowe kolumny: `title`, `text`, `subject`. Kolejnym etapem jest połączenie tych zbiorów w jedną tabelę danych. Na koniec dane tekstowe, oznaczone jako `X`, oraz etykiety, oznaczone jako `y`, są przygotowywane do dalszej analizy.

Następnym krokiem jest tworzenie reprezentacji kontekstowej, realizowane przez funkcję `get_embeddings`. Do dyspozycji mamy trzy możliwe opcje, przedstawione poniżej:

- **BERT**: funkcja `get_bert_embeddings` tokenizuje teksty przy użyciu `BertTokenizer`, a następnie generuje osadzenia w partiach (`batch_size=32`) za pomocą modelu `TFBertForSequenceClassification`.

- **RoBERTa**: funkcja `get_roberta_embeddings` tokenizuje teksty przy użyciu `RobertaTokenizer`, a następnie generuje osadzenia w partiach (`batch_size=32`) za pomocą modelu `TFRobertaForSequenceClassification`.
- **Sentence Transformers**: funkcja `get_transformer_embeddings` wykorzystuje model `all-MiniLM-L6-v2` do generowania osadzeń.

Alternatywnie, jeśli wybrano wektoryzację TF-IDF, funkcja `vectorize_data` przekształca dane tekstowe na wektor cech TF-IDF, z maksymalną liczbą cech ustawioną na `max_features=5000`, uwzględniając bigramy i usuwając stopwords.

Ostatnim krokiem jest podział danych na zbiory treningowe i testowe, realizowany przez jedną z poniższych funkcji:

- **split_data**: wykonuje klasyczny podział w proporcji 80:20 (z `test_size=0.2`) z zachowaniem losowości (`random_state=42`).
- **split_data_one_shot**: wybiera dokładnie jeden przykład na klasę do zbioru testowego, a resztę danych przypisuje do zbioru treningowego, co jest przydatne w scenariuszach z ograniczoną liczbą danych.
- **split_data_few_shot**: wybiera kilka przykładów na klasę (`few_shot_examples=5`), co pozwala na symulację scenariuszy z małą liczbą przykładów treningowych.

Wszystkie te kroki przygotowują dane do dalszego etapu przetwarzania klasyfikacyjnego, opisanego w kolejnych sekcjach, gdzie są wykorzystywane przez każdą metodę jako:

- **X_train** - tablica `ndarray` przechowująca cechy zbioru treningowego, takie jak wektory TF-IDF lub osadzenia kontekstowe z BERT, RoBERTa czy Sentence Transformers, wykorzystywane do trenowania modelu klasyfikacyjnego.
- **y_train** - lista lub tablica `ndarray` zawierająca etykiety binarne zbioru treningowego, gdzie 0 oznacza fałszywe wiadomości, a 1 oznacza prawdziwe.
- **X_test** - tablica `ndarray` przechowująca cechy zbioru testowego, w formacie analogicznym do `X_train`, używane do ewaluacji skuteczności modelu.
- **y_test** - lista lub tablica `ndarray` zawierająca etykiety binarne zbioru testowego, gdzie 0 oznacza fałszywe wiadomości, a 1 oznacza prawdziwe, służące jako punkt odniesienia dla predykcji modelu.

2.7. IMPLEMENTACJA WYBRANYCH NAUKOWYCH METOD

Celem niniejszego przeglądu jest analiza wybranych implementacji stosowanych w wykrywaniu fake newsów. Podzielono je na pięć głównych kategorii: **modele oparte na drzewach** (Tree-Based Models), **modele liniowe i statystyczne** (Linear and Statistical Models), **sieci neuronowe** (Neural Networks), **zespoły klasyfikatorów** (Ensemble Classifiers) oraz **modele zoptymalizowane** (Optimized Models). W każdej z tych kategorii

omówiono charakterystyczne cechy oraz innowacyjne modyfikacje w implementacjach, które przyczyniają się do poprawy skuteczności klasyfikacji. Pragniemy podkreślić, że wszystkie 16 omawianych podejść i implementacji bazują na 16 różnych metodach naukowych, stanowiących fundament analizy. Wybór tych konkretnych metod został podyktowany ich udokumentowaną skutecznością w literaturze naukowej, ze szczególnym uwzględnieniem publikacji z ostatnich lat, które koncentrują się na analizie danych tekstowych i detekcji dezinformacji. Metody te zostały wyselekcjonowane na podstawie ich zdolności do adresowania kluczowych wyzwań w tym obszarze, takich jak przetwarzanie danych, modelowanie złożonych zależności semantycznych oraz radzenie sobie z niezrównoważonymi zbiorami danych, charakterystycznymi dla problematyki fake newsów. Dodatkowo, istotnym czynnikiem wyboru była łatwość implementacji, dostępność propozycji kodu źródłowego w popularnych bibliotekach, gotowych implementacji, oraz programistyczna wygoda, ułatwiająca adaptację i testowanie tych metod w praktyce.

Dodatkowo, proponujemy traktować każde podejście, opisane w jednym z 16 artykułów naukowych, jako odrębną metodę, co ułatwi ich prezentację i wyjaśnienie. Jednocześnie podkreślamy, że autorskie metody, będą omówione w kolejnych rozdziałach, zostaną przedstawione jako metody numer 17, 18, 19 i 20, uzupełniając cel badania. Autorskie metody zostały opracowane na podstawie analizy 16 wspomnianych wyżej podejść naukowych, co pozwoliło na stworzenie skutecznych rozwiązań.

2.7.1. Modele oparte na drzewach

metoda12

metoda12 - Random Forest i CatBoost: ensemble learning z baggingiem i boostingiem. Metoda ta trenuje równoległe klasyfikatory Random Forest i CatBoost, oceniając ich predykcje na danych testowych za pomocą metryk takich jak dokładność, precyzja, recall, F1 i ROC-AUC.

W oryginalnym podejściu opisanym w artykule [1] proces klasyfikacji opiera się na ensemble learningu z baggingiem Random Forest i boostingiem CatBoost dla binarnej detekcji fałszywych wiadomości. Dane z zestawów ISOT Fake News i COVID-19 Fake News są oczyszczane: usuwanie stop-słów, lematyzacja za pomocą NLTK WordNetLemmatizer, tworząc wektory cech. Zbiór jest dzielony na 80% treningowy i 20% testowy, a modele Random Forest oraz CatBoost są trenowane na danych treningowych, z CatBoost monitorującym dane testowe. Wyniki są oceniane na zestawie testowym.

Nasza implementacja metoda12 zachowuje wspólne cechy tej metody opisanej w [1], opierając się na równoległym treningu Random Forest i CatBoost oraz ocenie metryk. Dane wejściowe, czyli `X_train` reprezentujące cechy treningowe oraz `X_test` odpowiadające cechom testowym, są przyjmowane jako tablice bez dodatkowego preprocessingu, co różni się od oczyszczania i kodowania TF-IDF w artykule, ale zachowuje ideę pra-

cy na wektorach cech. Proces zaczyna się od inicjalizacji `RandomForestClassifier` i `CatBoostClassifier`, co jest zgodne z konfiguracją z artykułu. Random Forest jest trenowany na `X_train` i `y_train`, generując predykcje `rf_y_pred` na `X_test`, podobnie jak w artykule. CatBoost jest trenowany na `X_train` i `y_train` z monitorowaniem `X_test` i `y_test`, generując predykcje `catboost_y_pred`, co odzwierciedla ustawienie `eval_set` z artykułu. Wyniki są oceniane za pomocą funkcji `evaluate_model`, obliczającej dokładność, precyzję, recall, F1 i ROC-AUC na podstawie `y_test`, co odpowiada metrykom z artykułu. Metoda zwraca słownik zawierający wytrenowane modele (`rf_classifier`, `catboost_classifier`) oraz ich metryki (`rf_metrics`, `catboost_metrics`), co umożliwia dalszą analizę, podobnie jak w artykule, gdzie porównywano wydajność modeli.

2.7.2. Sieci neuronowe

metoda1

`metoda1` - Ensemble sieci neuronowych: klasyfikacja binarna z różnorodnymi hiperparametrami. Metoda ta trenuje wiele modeli sieci neuronowych o różnych konfiguracjach hiperparametrów, łącząc ich przewidywania w ensemble, a wyniki ocenia na danych testowych za pomocą uśrednionych predykcji.

Nasza implementacja `metoda1` całkowicie opiera się na artykule [3], który opisuje trening ensemble składający się z 5 sieci neuronowych, z losowo wybieranymi hiperparametrami. Każdy model tworzony jest za pomocą funkcji `create_model`, budującej sieć sekwencyjną z trzema warstwami gęstymi (`units1`, `units2`, `units3`), z regularizacją L2, normalizacją (`BatchNormalization`), aktywacją `LeakyReLU`, warstwami Dropout i wyjściową warstwą z aktywacją `sigmoid` dla klasyfikacji binarnej. Model jest kompilowany z optymalizatorem Adam i funkcją straty `binary_crossentropy`. Trening każdego modelu odbywa się na danych `X_train` i `y_train` przez 20 iteracji, z walidacją na `X_test` i `y_test`, wykorzystując redukcję `ReduceLROnPlateau` o czynnik 0.5 przy stagnacji straty walidacyjnej przez 3 iteracji. Predykcje każdego modelu na `X_test` są zbierane i uśredniane, tworząc finalne przewidywania `predictions`. Metoda zwraca listę wytrenowanych modeli `models`, `X_test` oraz `y_test`, umożliwiając dalszą analizę.

metoda4

`metoda4` - Bi-LSTM, CNN i MLP z klasyfikacją sigmoidalną. Metoda ta wykorzystuje podejście ensemble, w którym Bi-LSTM, CNN i MLP przetwarzają dane wejściowe, a wyniki są łączone i klasyfikowane za pomocą funkcji sigmoidalnej.

Nasza implementacja `metoda4` zachowuje oryginalne cechy tego podejścia opisanego w [17], opierając się na ensemble learningu z Bi-LSTM, CNN i MLP, ale koncentruje się na klasyfikacji binarnej. Dane wejściowe, czyli `X_train` reprezentujące cechy treningowe oraz `X_test` odpowiadające cechom testowym, są skalowane za pomocą `StandardScaler`

i przekształcane na format trójwymiarowy (`X_train_reshaped`, `X_test_reshaped`), co różni się od osadzania wektorowego w artykule, ale zachowuje ideę przygotowania danych do sieci neuronowych. Proces zaczyna się od inicjalizacji warstwy wejściowej (Input) o dynamicznej długości sekwencji, przez którą dane przechodzą do trzech ścieżek: Bidirectional (LSTM) dla sekwencyjnego przetwarzania, Conv1D z MaxPooling1D dla wykrywania wzorców oraz Dense dla prostego przetwarzania cech.

Wyjścia są spłaszczane, łączone i przekazywane do warstwy Dense z aktywacją ReLU, a następnie do warstwy wyjściowej z aktywacją softmax Dense, co odpowiada podejściu w artykule. Model jest kompilowany z optymalizatorem Adadelta i trenowany przez 3 rodzaje próbek, używając `X_test_reshaped` i `y_test` do walidacji, podobnie jak w artykule, gdzie testowy zestaw LIAR był używany do oceny.

Metoda zwraca wytrenowany model (Model), `X_test_reshaped` oraz `y_test`, co umożliwia dalszą analizę, zgodnie z ideą oceny wyników w artykule.

metoda15

`metoda15` - Ensemble modeli Bi-LSTM: modeli zespołowe oparte na dwukierunkowej warstwie LSTM z tokenizacją i wyrównaniem długości sekwencji tekstowych.

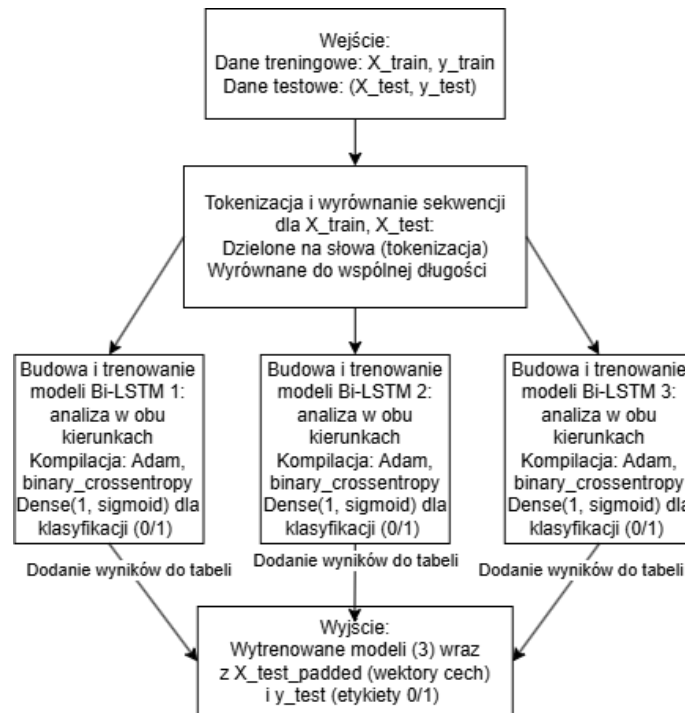
Dane wejściowe są najpierw dzielone na pojedyncze słowa (tokenizacja), a ich długości są wyrównywane do wspólnej wartości, aby pasowały do wymagań sieci. Następnie tworzona jest warstwa wejściowa, przez którą dane przechodzą do trzech modeli Bi-LSTM, które analizują tekst w obu kierunkach aby lepiej zrozumieć kontekst. Każdy model jest kompilowany z optymalizatorem Adam, używając funkcji straty `binary_crossentropy` do klasyfikacji binarnej. Wyniki z tych modeli są łączone i przetwarzane, a na końcu klasyfikowane za pomocą funkcji sigmoidalnej, która określa, czy wiadomość jest prawdziwa (1) czy fałszywa (0).

Modele są trenowane przez maksymalnie 20 cykli z partiami po 64 próbki, z danymi testowymi używanymi do sprawdzenia postępów. W trakcie trenowania stosowany jest mechanizm wczesnego zatrzymania (EarlyStopping), który monitoruje stratę na danych walidacyjnych i przywraca najlepsze wagi, jeśli wyniki przestają się poprawiać przez 2 cykle, co zapobiega przeuczeniu. Na koniec tworzona jest lista trzech wytrenowanych modeli Bi-LSTM. Wyniki są zwracane jako lista wytrenowanych modeli oraz przygotowane dane testowe.

Lista wytrenowanych modeli to trzy gotowe sieci neuronowe zdolne do przewidywania, czy wiadomość jest prawdziwa (1) czy fałszywa (0). Przygotowane dane testowe obejmują tablicę cech (`X_test`), która zawiera przetworzone teksty po tokenizacji i wyrównaniu, oraz tablicę etykiet (`y_test`) z wartościami 0 lub 1, służącymi do weryfikacji predykcji.

Dzięki analizie artykułu [19], powstała implementacja `metoda15`. Różnica pomiędzy naszą implementacją a oryginałem polega na tym, że artykuł opisuje ensemble Bi-LSTM z VGG-19 i mechanizmem uwagi do klasyfikacji multimodalnej (tekst i obrazy) z niestandar-

dową funkcją straty, podczas gdy nasza implementacja skupia się na klasyfikacji binarnej tekstu za pomocą Bi-LSTM bez dodatkowej warstwy wizualnej. Chcąc zaprezentować nasz pomysł, szczegóły implementacji są pokazane na podanym diagramie 2.1.



Rys. 2.1: Schemat metody 15.

2.7.3. Zespoły klasyfikatorów

metoda2

metoda2 - Stacking z GradientBoosting i MLP z Regresją Logistyczną. W artykule [14] się opiuje podejście stackingu, w którym modele bazowe generują predykcje prawdopodobieństw, a meta-klasyfikator na ich podstawie dokonuje ostatecznej klasyfikacji.

Nasza implementacja metoda2 zachowuje kluczowe wspólne cechy tego podejścia, opierając się na podobnej strukturze stackingu i wykorzystaniu ensemble learningu. W naszej wersji dane wejściowe, czyli `X_train` reprezentujące cechy treningowe oraz `X_test` odpowiadające cechom testowym. Proces zaczyna się od inicjalizacji modeli bazowych: `GradientBoostingClassifier` oznaczanego jako `boosted_tree` oraz `MLPClassifier` oznaczanego jako `neural_net`, co odpowiada użyciu Boosted Decision Tree i Neural Network w artykule.

Następnie generujemy predykcje dla danych treningowych, czyli `boosted_tree_preds_test` i `neural_net_preds_test`, obliczając prawdopodobieństwa klas odpowiednio od `boosted_tree` na podzbiorze zachowań społecznych (`social_behavior_features_test`) i `neural_net` na podzbiorze demograficznym (`demographic_features_test`), które są łączone w `combined_preds_test`, co jest

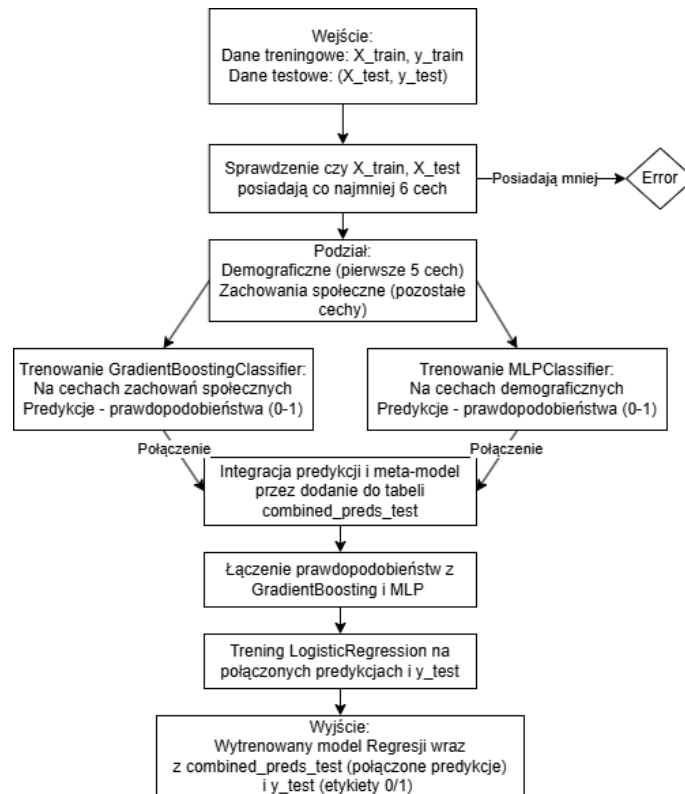
zgodne z podejściem artykułu, w którym predykcje modeli bazowych stają się meta-cechami. Te meta-cechy są używane do treningu meta-klasyfikatora `LogisticRegression`, oznaczonego jako `logistic_reg`, na `combined_preds_test` i `y_test`, co się bazuje na etapie treningu regresji logistycznej w artykule.

Modele bazowe `boosted_tree` i `neural_net` są trenowane na odpowiednich podzbiorach `X_train` (`social_behavior_features_train` i `demographic_features_train`) z `y_train`, podobnie jak w artykule, gdzie następuje trening na podzielonych danych.

Ostatecznie `logistic_reg` dokonuje predykcji na `combined_preds_test`, a wyniki mogą być oceniane na podstawie `y_test` (etykiety testowe), co jest wspólnym elementem z artykułem, gdzie stosowano metryki takie jak dokładność. Metoda zwraca `logistic_reg`, `combined_preds_test` oraz `y_test`, co umożliwia dalszą analizę.

Nasza implementacja różni się od podejścia w artykule [14], ponieważ zamiast `TwoClass Boosted Decision Tree` i `TwoClass Neural Network` stosuje `GradientBoostingClassifier` i `MLPClassifier`, co upraszcza strukturę modeli. Artykuł wykorzystuje preprocessing danych: czyszczenie, zastępowanie brakujących wartości modą.

Podczas gdy nasz kod działa na ogólnych danych bez preprocessingu, wymagając jedynie podziału na podzbiory demograficzne i społeczne. Wspólnym elementem jest stacking z Regresją Logistyczną jako meta-modelem, ale nasza wersja jest bardziej uniwersalna, tracąc specyficzność cech wiarygodności kluczowych w artykule. Szczegóły implementacji są pokazane na podanym diagramie 2.2.



Rys. 2.2: Schemat metody 2.

metoda3

metoda3 - Dwupoziomowy Voting Classifier: SVM, Naive Bayes, Decision Tree i Bagging/AdaBoost.

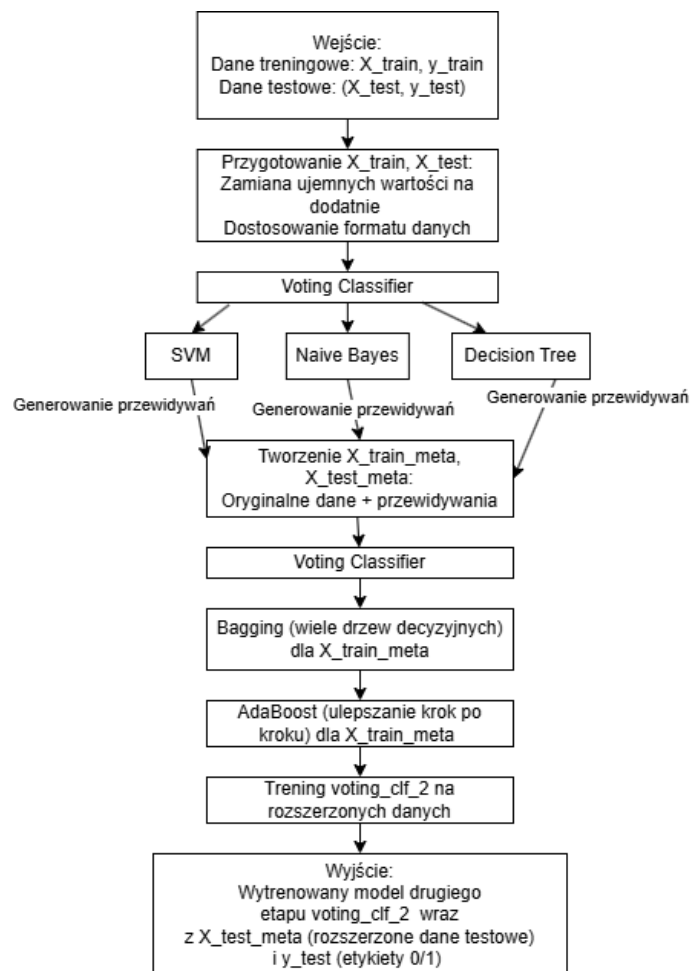
Nasza implementacja posiada trzy klasyfikatory bazowe: SVM SVC, Naive Bayes MultinomialNB oraz Decision Tree DecisionTreeClassifier. Klasyfikatory te są łączone przez VotingClassifier z miękkim głosowaniem, które generuje prawdopodobieństwa predykcji predict_proba dla X_train i X_test, tworząc meta-cechy: predictions_1_train i predictions_1_test. Te meta-cechy są łączone z oryginalnymi danymi, tworząc rozszerzone zbiory X_train_meta i X_test_meta, gdzie każda próbka zawiera oryginalne cechy oraz prawdopodobieństwa predykcji z pierwszego poziomu.

Następnie trenowany jest kolejny zespół VotingClassifier voting_clf_2, składający się z dwóch klasyfikatorów: BaggingClassifier oraz AdaBoostClassifier. Model ten jest dopasowywany do X_train_meta i y_train, łącząc predykcje z pierwszego poziomu z oryginalnymi cechami, aby uzyskać ostateczną decyzję klasyfikacyjną. Proces ten odzwierciedla ideę zespołowego uczenia z artykułu [20], gdzie stosowano voting classifier do agregacji wyników różnych modeli, choć w artykule nacisk położono na cechy językowe i word embedding.

Wyniki zwracane przez metoda3 obejmują: gotowy model drugiego poziomu

(`voting_clf_2`), czyli wytrenowany `VotingClassifier` z ustawieniami i wagami, meta-cechy testowe (`X_test_meta`), które łączą oryginalne cechy `X_test` z predykcjami pierwszego poziomu, oraz etykiety testowe (`y_test`) z wartościami 0 lub 1, umożliwiające ocenę skuteczności modelu.

Główna różnica między naszą implementacją a artykułem dotyczy podejścia do danych i struktury klasyfikacji - artykuł opiera się na ekstrakcji cech językowych i word embedding (TF-IDF, Count Vectorizer) z trzema zestawami LFS, podczas gdy nasza implementacja działa na ogólnych danych, wykorzystując predykcje pierwszego poziomu jako meta-cechy, bez preprocessingu tekstowego. Wspólnym elementem jest użycie voting classifiera i ensemble learningu z modelami takimi jak SVM, Naive Bayes czy Decision Tree, co zwiększa dokładność. Nasza implementacja jest bardziej uniwersalna, ale traci specyficzność cech tekstowych, kluczowych w artykule. Szczegóły implementacji, w tym przepływ danych przez klasyfikatory przedstawia diagram 2.3.



Rys. 2.3: Schemat metody 3.

metoda5

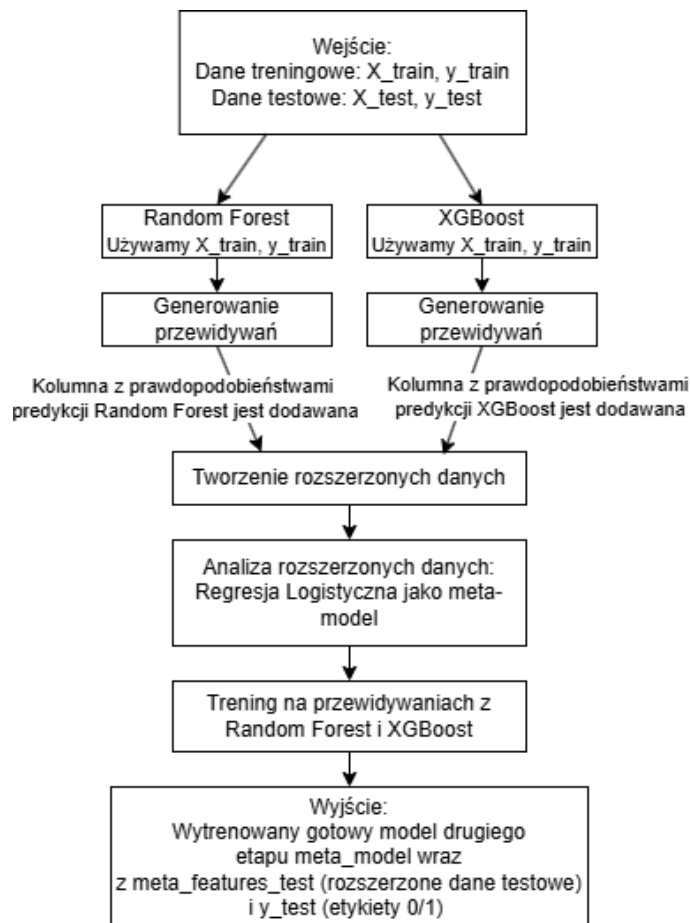
metoda5 - Dwuwarstwowy Stacking Classifier z Random Forest, XGBoost i regresją logistyczną: Nasza implementacja stosuje podejście stackingowe do klasyfikacji, zgodnie z ideami ensemble learningu opisanymi w artykule [7], gdzie różne modele maszynowe, takie jak Decision Tree, Random Forest i Extra Tree Classifier, są łączone w celu poprawy dokładności wykrywania fałszywych wiadomości.

Nasza implementacja w pierwszym etapie ma trenowanie dwóch modeli bazowych: Random Forest RandomForestClassifier oraz XGBoost XGBClassifier. Modele te są dopasowywane do `X_train` i `y_train`, generując prawdopodobieństwa predykcji (`predict_proba`) dla klasy pozytywnej, co tworzy nowe cechy dla meta-modelu: `rf_preds_train` i `xgb_preds_train`. Te predykcje są łączone w tablicę danych `meta_features_train` z kolumnami `rf_preds` i `xgb_preds`, zawierającą dwie cechy na próbkę.

W drugim etapie meta-model, czyli regresja logistyczna (LogisticRegression), jest trenowany na `meta_features_train` i `y_train`, ucząc się łączenia predykcji modeli bazowych w celu uzyskania ostatecznej decyzji. Dla danych testowych generowane są predykcje modeli bazowych (`rf_preds_test`, `xgb_preds_test`), które tworzą `meta_features_test`. W artykule [7] podobnie jak w naszej implementacji różne klasyfikatory są łączone poprzez agregację ich wyników, choć w artykule stosowano bagging zamiast stackingu.

Wyniki zwracane przez metoda5 obejmują: gotowy meta-model (`meta_model`), czyli wytrenowaną regresję logistyczną z ustawieniami i wagami, cechy testowe (`meta_features_test`), które łączą predykcje Random Forest i XGBoost dla `X_test` oraz etykiety testowe (`y_test`) z wartościami 0 lub 1, umożliwiające ocenę modelu. Struktura ta pozwala na analizę skuteczności modelu, podobnie jak w artykule, gdzie oceniano dokładność na zbiorach ISOT i Liar.

metoda5 różni się od podejścia opisanego w artykule [7]. W artykule autorzy skupili się na ekstrakcji 26 cech tekstowych z danych tekstowych, które następnie klasyfikowano za pomocą ensemble opartego na baggingu z Decision Tree, Random Forest i Extra Tree Classifier. Nasza implementacja nie wymaga ekstrakcji specyficznych cech tekstowych, działając na ogólnych danych wejściowych (`X_train`, `X_test`), i wykorzystuje stacking z Random Forest i XGBoost jako modelami bazowymi oraz regresją logistyczną jako meta-modelem, zamiast baggingu. Wspólną cechą jest zastosowanie ensemble learningu do poprawy dokładności poprzez łączenie różnych klasyfikatorów. Nasza metoda jest bardziej uniwersalna, ponieważ nie zakłada określonego typu danych, ale traci specyficzność cech tekstowych, które w artykule znacząco poprawiły wyniki po ekstrakcji cech. Artykuł podkreśla znaczenie preprocessingu: tokenizacja, usuwanie stop-words, stemming z użyciem NLTK, czego nasza implementacja nie obejmuje, koncentrując się na warstwie modelowania. Szczegóły implementacji przedstawia diagram 2.4.



Rys. 2.4: Schemat metody 5.

metoda6

metoda6 - Weighted Convolutional Neural Network Ensemble (WCNNE) z AdaBoost i opcjonalnymi osadzeniami BERT: Nasza implementacja stosuje podejście zespołowe do klasyfikacji sentymentów, zgodne z ideami ensemble learningu opisanymi w artykule [6], gdzie różne modele, w tym konwolucyjne sieci neuronowe (CNN), są łączone w celu poprawy dokładności analizy sentymentów, szczególnie na danych tekstowych.

Nasza implementacja w metoda6 rozpoczyna się od otrzymania danych treningowych (X_{train} , y_{train}) i testowych (X_{test} , y_{test}). Dane tekstowe w X_{train} i X_{test} są przekształcane na osadzenia za pomocą pretrenowanego modelu BERT (`bert-base-uncased`), co pozwala na lepsze zrozumienie semantyki tekstu, podobnie jak w artykule, gdzie BERT jest wykorzystywany do ekstrakcji cech tekstowych. Osadzenia są generowane z użyciem `BertTokenizer` i `TFBertModel`, tworząc trójwymiarowe tablice o wymiarach: liczba próbek, maksymalna długość sekwencji, wymiar osadzenia. Następnie dane są skalowane za pomocą `StandardScaler`, tworząc X_{train_scaled} i X_{test_scaled} , aby zapewnić normalizację wymaganą przez modele CNN, co odpowiada standardowemu preprocessingowi w artykule. Określa się kształt wejścia `input_shape`, `embedding_dim` na podstawie wymiarów X_{train_scaled} oraz liczbę klas

(`num_classes`) na podstawie unikalnych wartości w `y_train`, umożliwiając elastyczne dostosowanie modelu do klasyfikacji binarnej lub wieloklasowej.

W kolejnym etapie definiuje się klasyfikator bazowy w postaci konwolucyjnej sieci neuronowej (CNN) zaimplementowanej w metodzie `create_cnn_model`, zgodnej z konfiguracją z artykułu. Model CNN składa się z dwóch warstw `Conv1D`, przeplatanych warstwami `MaxPooling1D`, warstwy `Dropout` dla regularyzacji oraz warstwy `Flatten` i `Dense` z aktywacją sigmoidalną dla klasyfikacji binarnej, skompilowanej z optymalizatorem `Adam`. Klasyfikator CNN jest opakowany w `KerasClassifier`. Następnie tworzony jest klasyfikator zespołowy `WCNNE` z użyciem `AdaBoostClassifier`, który łączy trzy instancje CNN z szybkością uczenia `learning_rate=0.01`, co odpowiada strukturze `WCNNE` opartej na `AdaBoost` opisanej w [6]. Model `model` jest trenowany na `X_train_scaled` i `y_train`, co odzwierciedla sekwencyjne uczenie słabych klasyfikatorów w `AdaBoost` dla poprawy wydajności.

Wyniki są zwracane w następujący sposób: gotowy klasyfikator zespołowy (`model`), zawierający wytrenowane CNN i ustawienia `AdaBoost`, dane testowe (`X_test`), które są osadzeniami oraz etykiety testowe (`y_test`) z wartościami 0, 1, umożliwiające ocenę modelu.

metoda8

`metoda8` - Voting Classifier z Random Forest i XGBoost: probabilistyczne głosowanie predykcji.

W artykule [5] omówiono różne strategie ensemble learningu, takie jak bagging, boosting oraz głębokie modele neuronowe, stosowane w celu poprawy generalizacji modeli uczenia maszynowego, w tym Random Forest oparty na baggingu, modele oparte na boostingu, takie jak XGBoost, oraz głębokie sieci neuronowe, takie jak DBN, które zwiększają wydajność dzięki hierarchicznemu uczeniu cech, osiągając lepsze wyniki w klasyfikacji dzięki różnorodności predykcji.

Nasza implementacja w `metoda8` całkowicie jest podobna do wspomnianej z [5], realizując podejście zespołowe do klasyfikacji binarnej lub wieloklasowej poprzez połączenie Random Forest, XGBoost oraz głębokiej sieci neuronowej (DBN) w klasyfikatorze Voting Classifier z miękkim głosowaniem, co w pełni odpowiada strategiom ensemble learningu opisanym w artykule, szczególnie w aspekcie włączenia głębokich modeli. Proces rozpoczyna się od przygotowania danych treningowych (`X_train`, `y_train`) i testowych (`X_test`, `y_test`). Jeśli `X_train` lub `X_test` są macierzami rzadkimi, konwertuje się je na macierze gęste, co jest zgodne z potrzebą odpowiedniego formatowania danych w artykule, gdzie różne zestawy danych wymagały preprocessingu.

Następnie dane są skalowane za pomocą `StandardScaler`, tworząc `X_train_scaled` i `X_test_scaled`, aby zapewnić normalizację wymaganą przez głębokie modele neuronowe, co odzwierciedla standardowe praktyki preprocessingu dla DBN. Określa się

liczbę cech (`input_dim`) na podstawie `X_train_scaled.shape[1]` oraz liczbę klas (`num_classes`) na podstawie unikalnych wartości w `y_train`, co umożliwia dynamiczne dostosowanie modelu DBN do problemu klasyfikacji. Definiuje się trzy klasyfikatory bazowe: `RandomForestClassifier` z 100 estymatorami oznaczany jako `rf`, oparty na strategii baggingu; `XGBClassifier` z 100 estymatorami, oznaczany jako `xgb`, oparty na boostingu; oraz `KerasClassifier` opakowujący model DBN zdefiniowany w `create_dbn_model`, oznaczany jako `dbn`, z warstwami z aktywacją ReLU, warstwami Dropout dla regularyzacji oraz warstwą wyjściową z aktywacją sigmoidalną dla klasyfikacji binarnej lub softmax dla wieloklasowej. Wybór tych klasyfikatorów dokładnie pokrywa się z omówieniem baggingu, boostingu i głębokich modeli w artykule jako kluczowych technik ensemble learningu.

Klasyfikatory są łączone w `VotingClassifier`, oznaczany jako `ensemble_model`, z miękkim głosowaniem, które uśrednia prawdopodobieństwa predykcji z `rf`, `xgb` i `dbn`, podobnie jak w artykule strategie ensemble łączyły predykcje dla poprawy wydajności. Model `ensemble_model` jest trenowany na `X_train_scaled` i `y_train`, co odpowiada etapowi uczenia w artykule, gdzie modele ensemble, w tym głębokie, były trenowane na różnych zbiorach danych po odpowiednim preprocessingu. Po wytrenowaniu metoda zwraca `ensemble_model`, `X_test` oraz `y_test`, gdzie `y_test` zawiera etykiety binarne (0 lub 1) lub wieloklasowe, umożliwiając dalszą ocenę modelu, co jest zgodne z praktyką oceny modeli w [5].

metoda9

`metoda9` - Voting Classifier z Logistic Regression, Random Forest i SVC oraz porównanie modeli zespołowych.

W artykule [2] opisano podejście oparte na ensemble learning do wykrywania fałszywych wiadomości, stosując różne techniki zespołowe, w tym Voting Classifier z `LogisticRegression`, `RandomForestClassifier` oraz innymi modelami, takimi jak KNN lub LSVM, a także Bagging i Boosting, w tym AdaBoost i XGBoost, oceniane za pomocą metryk takich jak dokładność, precyzja, recall i F1-score na czterech zbiorach danych, osiągając średnią dokładność 92.25% dla metod zespołowych.

Nasza implementacja w `metoda9` zachowuje te kluczowe cechy, opierając się na podobnym podejściu ensemble learningu do klasyfikacji binarnej fałszywych wiadomości, z wykorzystaniem kombinacji modeli i ich predykcji dla poprawy dokładności. W artykule dane są przygotowywane z użyciem narzędzia LIWC do ekstrakcji cech językowych, co w naszym procesie odpowiada treningowi na danych `X_train` (cechy treningowe) i `y_train` (etykiety treningowe), które są już w odpowiedniej formie wektorowej, umożliwiając bezpośrednie zastosowanie modeli. Proces rozpoczyna się od zdefiniowania modeli bazowych: `LogisticRegression` oznaczanego jako `log_reg`, `RandomForestClassifier` oznaczanego jako `rf` oraz `KNeighborsClassifier` oznaczanego jako `knn`, które pokrywają się z wyborem modeli jak w artykule, tworząc podstawę dla uczenia zespołowego.

Dodatkowo stosujemy `BaggingClassifier` z `rf` jako estymatorem, oznaczany jako bagging, oraz `AdaBoostClassifier`, oznaczany jako boosting, co jest zgodne z technikami Bagging i Boosting (AdaBoost) opisanymi w [2], wzmacniając ideę ensemble learningu. Modele te są łączone w `VotingClassifier`, oznaczanym jako voting, co pozwala na integrację predykcji prawdopodobieństw z każdego modelu, podobnie jak w artykule `Voting Classifier` łączył wyniki modeli dla uzyskania wysokiej dokładności. Trening wszystkich modeli, w tym `rf`, bagging, boosting i voting, odbywa się na `X_train` i `y_train`, co odpowiada etapowi uczenia w artykule, gdzie różne algorytmy były trenowane na danych z cechami językowymi.

Po wytrenowaniu każdy model generuje predykcje `y_pred` na `X_test` (cechy testowe), a wyniki są oceniane za pomocą `accuracy_score`, `roc_auc_score` (jeśli model wspiera predykcje prawdopodobieństw) oraz `classification_report` na podstawie `y_test` (etykiety testowe z wartościami 0 lub 1), co jest wspólnym elementem z artykułem, gdzie stosowano metryki takie jak dokładność, precyzja, recall i F1-score do oceny wydajności. Metoda zwraca model (najlepszy model), `X_test` oraz `y_test`, umożliwiając dalszą analizę, podobnie jak w artykule wyniki były dostępne do porównania na różnych zbiorach danych.

metoda10

`metoda10` - `Voting Classifier` z `MLP`, `Logistic Regression` i `XGBoost` z miękkim głosowaniem.

W artykule [10] jest zaproponowany `Voting Classifier` z trzema najlepszymi klasyfikatorami wybranymi po walidacji krzyżowej: `MLPClassifier`, `LogisticRegression` i `XGBClassifier`. Modele te są łączone z miękkim głosowaniem (`voting='soft'`) na podstawie prawdopodobieństw, osiągając dokładność 94.47% na danych tekstowych po preprocessingu.

Nasza implementacja w `metoda10` zachowuje kluczowe wspólne cechy tego podejścia, wykorzystując identyczne modele i strukturę ensemble do klasyfikacji binarnej na podstawie danych tekstowych. W artykule modele są trenowane na danych po preprocessingu, co znajduje swoje odzwierciedlenie w naszym procesie, gdzie dane wejściowe `X_train` (cechy treningowe) i `y_train` (etykiety treningowe) są bezpośrednio używane do treningu, choć w naszym kodzie pomijamy preprocessing, korzystając z wspólnego procesu przygotowania danych dla wszystkich metod, co czyni podejście bardziej uniwersalnym, ale zachowuje zgodność z ideą klasyfikacji na przygotowanych danych.

Proces rozpoczyna się od inicjalizacji tych samych klasyfikatorów co w artykule: `MLPClassifier` oznaczanego jako `mlp`, `LogisticRegression` oznaczanego jako `log_reg` oraz `XGBClassifier` oznaczanego jako `xgb`, które są zgodne z wyborem najlepszych modeli w [10], tworząc podstawę dla uczenia zespołowego. Te modele są następnie łączone w `VotingClassifier`, oznaczanym jako `voting_clf`, z miękkim głosowaniem

(`voting='soft'`), co pozwala na integrację predykcji prawdopodobieństw z każdego modelu, dokładnie tak jak w artykule Voting Classifier opierał się na kombinacji wyników dla osiągnięcia wysokiej dokładności.

Trening `voting_clf` odbywa się na `X_train` i `y_train`, co odpowiada etapowi uczenia w artykule, gdzie modele były trenowane na danych tekstowych po preprocessingu, a w naszym przypadku dane są już w odpowiedniej formie dzięki wspólnemu pipeline. Po wytrenowaniu `voting_clf` jest gotowy do predykcji na `X_test` (macierz cech testowych), a metoda zwraca `voting_clf`, który zawiera ustawienia i wyniki uczenia, wraz z `X_test` oraz `y_test` (wektor etykiet testowych z wartościami 0 lub 1), co umożliwia ocenę jakości modelu, podobnie jak w artykule wyniki klasyfikacji służyły do weryfikacji skuteczności, osiągając wspomnianą dokładność.

Mamy dokładnie to samo podejście co w [10], opierając się na Voting Classifier z identycznymi modelami `MLPClassifier`, `LogisticRegression` i `XGBClassifier` oraz `soft` głosowaniem, z tą różnicą, że pomijamy preprocessing w kodzie na rzecz uniwersalnego przygotowania danych, co nie zmienia istoty ensemble methods opisanych w artykule.

metoda11

`metoda11` - Voting Classifier z Random Forest, Logistic Regression i AdaBoost po selekcji cech.

W artykule [15] opisano zastosowanie ensemble methods, w tym Voting Classifier z `RandomForestClassifier`, `LogisticRegression` i `AdaBoostClassifier`, do wykrywania fałszywych wiadomości na podstawie cech tekstowych, osiągając wysoką dokładność dzięki kombinacji predykcji.

Nasza implementacja `metoda11` całkiem reprezentuje to podejście, opierając się na identycznym zestawie modeli i idei łączenia ich predykcji w celu poprawy klasyfikacji binarnej fałszywych wiadomości. W artykule wykorzystano cechy tekstowe do treningu modeli, co znajduje swoje odzwierciedlenie w naszym procesie, gdzie dane wejściowe `X_train` (cechy treningowe) i `X_test` (cechy testowe) są poddawane normalizacji za pomocą `MinMaxScaler`, tworząc `X_train_scaled` i `X_test_scaled`, aby zapewnić nieujemne wartości wymagane do dalszej selekcji cech, co jest wspólnym krokiem przygotowania danych do klasyfikacji.

Następnie stosujemy selekcję cech za pomocą `SelectKBest` z testem Chi-kwadrat, przekształcając

`X_train_scaled` w `X_train_selected` oraz `X_test_scaled` w `X_test_selected`, co pozwala skoncentrować się na najważniejszych cechach, podobnie jak w artykule dane tekstowe były optymalizowane przed klasyfikacją. Proces kontynuowany jest przez zdefiniowanie tych samych modeli bazowych co w artykule: `RandomForestClassifier` oznaczanego jako `rf`, `LogisticRegression` oznaczanego jako `lr` oraz `AdaBoostClassifier`

oznaczanego jako `adb`, które są zgodne z wyborem modeli w [15], tworząc podstawę dla uczenia zespołowego.

Te modele są następnie łączone w `VotingClassifier`, oznaczanym jako `voting_clf`, z miękkim głosowaniem (`voting='soft'`), co pozwala na integrację predykcji prawdopodobieństw z każdego modelu, dokładnie tak jak w artykule `ensemble methods` opierały się na kombinacji wyników dla zwiększenia dokładności. Trening `voting_clf` odbywa się na `X_train_selected` i `y_train` (etykiety treningowe), co odpowiada etapowi uczenia w artykule, gdzie modele były trenowane na przygotowanych danych tekstowych.

Po wytrenowaniu `voting_clf` jest gotowy do predykcji na `X_test_selected`, a metoda zwraca `voting_clf`, który zawiera ustawienia i wyniki uczenia, wraz z przetworzonymi danymi testowymi `X_test_selected` w formie macierzy oraz `y_test` (etykiety testowe jako wektor z wartościami 0 i 1), co umożliwia ocenę jakości modelu, podobnie jak w artykule wyniki klasyfikacji służyły do weryfikacji skuteczności. Mamy dokładnie to samo podejście co w [15], opierając się na `Voting Classifier` z tymi samymi modelami bazowymi i miękkim głosowaniem, z dodaną normalizacją i selekcją cech jako częścią naszego pipeline, co wzmacnia zgodność z ideą `ensemble methods` opisaną w artykule.

metoda13

`metoda13` - Stacking z `SMOTETomek`, wykorzystujący `Random Forest`, `Gradient Boosting`, `XGBoost` i `Logistic Regression` jako meta-model.

`Metoda13` trenuje model zespołowy stacking z wykorzystaniem `Random Forest`, `Gradient Boosting` i `XGBoost` jako klasyfikatorów bazowych, z `Logistic Regression` jako meta-ucznem, po uprzednim zbalansowaniu danych za pomocą `SMOTETomek`. Wyniki są oceniane na danych testowych za pomocą raportu klasyfikacji.

Nasza implementacja `metoda13` opiera się na `ensemble learningu` typu stacking, wspartym metodą `SMOTETomek` do obsługi problemu niezrównoważonych klas. Dane wejściowe, czyli `X_train` (cechy treningowe) i `y_train` (etykiety treningowe), są najpierw zbalansowane za pomocą `SMOTETomek`, co łączy nadpróbkowanie `SMOTE` z usuwaniem próbek metodą `Tomek Links`, tworząc zbalansowany zbiór `X_train_resampled` i `y_train_resampled`.

Następnie definiowane są trzy klasyfikatory bazowe: `RandomForestClassifier`, `GradientBoostingClassifier` oraz `XGBClassifier`. Te modele są łączone w `StackingClassifier` z `LogisticRegression` jako meta-uczeniem, stosując 5-krotną walidację krzyżową. Model jest trenowany na zbalansowanych danych, a predykcje `y_pred` generowane na `X_test` są oceniane za pomocą `classification_report`, który dostarcza metryki takie jak precyzja, `recall` i `F1` dla `y_test`.

Metoda zwraca wytrenowany model `stacking_model`, `X_test` oraz `y_test`, umożliwiając dalszą analizę. Nasza implementacja zachowuje kluczowe cechy podejścia opisanego w artykule [8], koncentrując się na stackingu i obsłudze niezrównoważonych klas.

metoda14

metoda14 - Stacking z Random Forest, AdaBoost, SVC i Logistic Regression jako meta-model.

Metoda14 opisana w artykule [9] trenuje model zespołowy stacking, wykorzystujący Random Forest, AdaBoost i SVC jako klasyfikatory bazowe, z Logistic Regression jako meta-ucznem.

Nasza implementacja metoda14 bazuje na koncepcji ensemble learningu opisanej w artykule [9], w szczególności na wykorzystaniu modeli takich jak AdaBoost i SVM, ale stosuje bardziej zaawansowane podejście stackingowe. Dane wejściowe `X_train` (cechy treningowe) i `y_train` (etykiety treningowe) oraz `X_test` (cechy testowe) i `y_test` (etykiety testowe) zakładają, że tekst został wcześniej przetworzony na osadzenia wektorowe. Funkcja `preprocess_text` oczyszcza tekst, usuwając URL-e, znaki specjalne, pojedyncze litery, liczby, konwertując na małe litery i stosując stemming za pomocą `SnowballStemmer`.

Najpierw definiowane są klasyfikatory bazowe: `RandomForestClassifier`, `AdaBoostClassifier` oraz `SVC`. Następnie `StackingClassifier` łączy je z `LogisticRegression` jako metauczeniem, stosując 5-krotną walidację krzyżową. Model jest trenowany na `X_train` i `y_train`, a po zakończeniu zwraca wytrenowany `stack_model`, `X_test` i `y_test`, umożliwiając dalszą ewaluację. Nasza implementacja nie generuje cech w kodzie i pomija bezpośrednią ocenę wyników, ponieważ `X_test` i `y_test` zostaną użyte do predykcji i ewaluacji poza funkcją.

Metoda16

Metoda16 - Stacking z meta-cechami: Random Forest, SVC z Logistic Regression. Metoda ta wykorzystuje podejście stackingu, w którym modele bazowe generują meta-cechy, a meta-klasyfikator na ich podstawie dokonuje ostatecznej predykcji.

W oryginalnym podejściu opisanym w artykule [12] proces klasyfikacji przebiega w kilku etapach. Najpierw dane tekstowe są przetwarzane za pomocą biblioteki NLTK, co obejmuje usuwanie stop-słów, stemming i lematyzację, a następnie konwertowane na cechy numeryczne za pomocą metod takich jak TF-IDF i `CountVectorizer`. Następnie stosuje się ensemble learning w formie stackingu, gdzie modele bazowe, takie jak Random Forest i SVM, generują predykcje prawdopodobieństw klas dla danych treningowych za pomocą walidacji krzyżowej. Te predykcje są używane jako meta-cechy do treningu meta-klasyfikatora, którym jest regresja logistyczna. Modele bazowe są następnie trenowane na pełnym zbiorze treningowym, a meta-cechy dla danych testowych są generowane na podstawie ich predykcji. Ostatecznie meta-klasyfikator dokonuje predykcji na meta-cechach testowych, a wyniki są oceniane za pomocą metryk takich jak dokładność i raport klasyfikacji.

Nasza implementacja metoda16 zachowuje kluczowe wspólne cechy tego podejścia, opierając się na podobnej strukturze stackingu i wykorzystaniu ensemble learningu. W naszej wersji dane wejściowe, czyli

`X_train` reprezentujące cechy treningowe oraz `X_test` odpowiadające cechom testowym, są już przygotowane jako osadzenia wektorowe w ramach wspólnego pipeline, co różni się od preprocessingu NLTK i metod TF-IDF/CountVectorizer stosowanych w artykule, ale zachowuje ideę przekształcenia danych do postaci numerycznej przed klasyfikacją. Proces zaczyna się od inicjalizacji modeli bazowych: `RandomForestClassifier` oznaczanego jako `rf_model` oraz `SVC` z liniowym jądrem oznaczanego jako `svc_model`, co odpowiada użyciu Random Forest i SVM w artykule.

Następnie generujemy meta-cechy dla danych treningowych, czyli `meta_features_train`, poprzez 5-krotną walidację krzyżową z rozwarstwieniem (`StratifiedKfold`), gdzie `rf_preds_train` i `svc_preds_train` to predykcje prawdopodobieństw odpowiednio od `rf_model` i `svc_model` na `X_train` i `y_train` (etykiety treningowe), łączone w macierz `meta_features_train`, co jest zgodne z podejściem artykułu, w którym predykcje z walidacji krzyżowej stają się meta-cechami. Te meta-cechy są używane do treningu meta-klasyfikatora `LogisticRegression`, oznaczonego jako `meta_model`, na `meta_features_train` i `y_train`, co odzwierciedla etap treningu regresji logistycznej w artykule.

Po tym kroku modele bazowe `rf_model` i `svc_model` są trenowane na pełnym zestawie `X_train` i `y_train`, podobnie jak w artykule, gdzie następuje trening na całym zbiorze treningowym. Dla danych testowych generujemy meta-cechy `meta_features_test`, obliczając `rf_preds_test` i `svc_preds_test` jako predykcje prawdopodobieństw od `rf_model` i `svc_model` na `X_test`, a następnie łącząc je w macierz `meta_features_test`, co odpowiada generowaniu meta-cech testowych w artykule. Ostatecznie `meta_model` dokonuje predykcji `y_pred` na `meta_features_test`, a wyniki są oceniane za pomocą `classification_report` i `accuracy_score` na podstawie `y_test` (etykiety testowe), co jest wspólnym elementem z artykułem, gdzie również stosowano metryki takie jak dokładność i raport klasyfikacji. Metoda zwraca `meta_model`, `meta_features_test` oraz `y_test`, co umożliwia dalszą analizę.

2.7.4. Modele zoptymalizowane

Metoda7

Metoda7 - Random Forest z Monte Carlo Dropout i heurystyczną obróbką post-procesową. Metoda7 opiera się na Random Forest jako bazowym klasyfikatorze.

Nasza implementacja w `metoda7` w pełni bazuje na tym podejściu, zachowując jego kluczowe elementy, ale dostosowuje je do dostępnych zasobów. Zamiast ensemble transformerów i NewsBERT, wykorzystujemy pojedynczy model Random Forest `RandomForestClassifier`, co wynika z faktu, że dane wejściowe `X_train` (cechy treningowe) i `X_test` (cechy testowe) są już przekształcone w osadzenia wektorowe, eliminując potrzebę tokenizacji czy generowania cech przez modele językowe.

Proces rozpoczyna się od przygotowania danych, gdzie `X_train` i `y_train` (etykiety treningowe) są konwertowane na `train_embeddings` i `train_labels`, a `X_test` oraz `y_test` (etykiety testowe) na `test_embeddings`. Dane są standaryzowane przed dalszym przetwarzaniem. Model Random Forest jest trenowany na `train_embeddings` i `train_labels`, co odpowiada etapowi uczenia w artykule, choć bez użycia transformerów. Predykcje na `test_embeddings` są generowane za pomocą symulacji Monte Carlo Dropout (`monte_carlo_dropout_inference`), która w 50 iteracjach próbkując 80% drzew, tworzy `mean_preds` (średnie prawdopodobieństwa), `uncertainty` (niepewność) oraz etykiety `pred_labels`. Ten krok odwzorowuje oszacowanie niepewności z artykułu, dostosowane do Random Forest.

Wprowadzamy metaatrybuty w `df` jako kolumnę "source", domyślnie ustawioną na "unknown", które są przetwarzane w `compute_meta_attribute_probabilities` do `attribute_probs`. Te dane są wykorzystywane w heurystycznej obróbce post-procesowej z progiem 0.9, korygując `pred_labels` na `final_predictions`, podobnie jak moduł SFTN w artykule. Jednak wpływ meta-atrybutów jest ograniczony z powodu ich uproszczenia w porównaniu do różnorodnych źródeł w oryginalnym podejściu.

Wyniki modelu są oceniane na `y_test` za pomocą `classification_report`. Funkcja zwraca `model`, `X_test` i `y_test`, umożliwiając dalszą analizę, co jest zgodne z podejściem artykułu.

2.8. OPIS ZESTAWÓW DANYCH UŻYTYCH W EKSPERYMENTACH

W naszych badaniach postanowiliśmy skupić się na trzech zbiorach danych: BuzzFeed News Dataset, ISOT Fake News Dataset oraz WELFake Dataset. Wybór tych zbiorów został podyktowany ich popularnością w literaturze naukowej, dostępnością odpowiednich danych oraz ich przydatnością do zadania wykrywania fałszywych informacji. Warto również wspomnieć, że rozważaliśmy użycie zbioru LIAR Dataset, który jest często wykorzystywany w badaniach nad fałszywymi informacjami, szczególnie w kontekście analizy wypowiedzi politycznych. Jednak LIAR Dataset nie został ostatecznie wybrany, ponieważ brakowało w nim kluczowych kolumn, takich jak pełne teksty artykułów czy metadane dotyczące źródeł publikacji, co czyniło go nieodpowiednim do naszych potrzeb eksperymentalnych, które wymagały szczegółowych danych tekstowych i kontekstowych.

2.8.1. ISOT Fake News Dataset

Zbiór danych dotyczących fałszywych wiadomości ISOT jest jednym z najczęściej wykorzystywanych zbiorów danych w badaniach nad wykrywaniem fałszywych wiadomości, szczególnie w analizie tekstów i klasyfikacji wiadomości. Ten zbiór jest popularny ze

względem na swój duży rozmiar, zrównoważoną strukturę. Szczegółowy opis użycia tego zbioru danych jest wspomniany w podanym artykule [11], w którym autorzy analizują różne podejścia do wykrywania fake newsów, w tym metody oparte na uczeniu maszynowym i głębokim uczeniu, takie jak LSTM lub modele hybrydowe, co potwierdza znaczenie starannie przygotowanych zbiorów danych, takich jak ISOT, w tego typu badaniach. Dlatego też ISOT stał się podstawowym zbiorem danych, w którym przeprowadzamy większość naszych eksperymentów i służy jako punkt odniesienia umożliwiający porównywanie wyników z innymi zbiorami danych.

Zbiór ISOT jest szczególnie ceniony za swoje cechy rzeczywistych scenariuszy dezinformacji, co pozwala na testowanie modeli w warunkach zbliżonych do tych, z jakimi spotykamy się w mediach społecznościowych czy portalach informacyjnych. W kontekście badań nad fałszywymi wiadomościami, takich jak te opisane w [11], ISOT umożliwia analizę zarówno lingwistycznych, jak i kontekstowych cech fałszywych wiadomości, co czyni go idealnym narzędziem do rozwijania zaawansowanych algorytmów wykrywania dezinformacji.

Struktura zbioru danych

ISOT Fake News Dataset składa się z dwóch głównych kategorii danych:

- **Fake News:** Zawiera artykuły sklasyfikowane jako fałszywe informacje, których celem jest wprowadzanie w błąd lub manipulacja opinią publiczną. Źródła tych artykułów pochodzą głównie z witryn internetowych znanych z publikowania niesprawdzonych, sensacyjnych lub celowo wprowadzających w błąd treści. Często charakteryzują się emocjonalnym językiem, prowokacyjnymi nagłówkami i brakiem wiarygodnych źródeł.
- **True News:** Zawiera artykuły uznane za prawdziwe, publikowane przez renomowane agencje informacyjne, takie jak Associated Press, Reuters czy BBC. Są to treści oparte na rzetelnym dziennikarstwie, w których stosuje się wiarygodne źródła i odpowiednie standardy edytorskie.

Szczegóły struktury danych

Każdy rekord w zbiorze ISOT zawiera następujące pola:

- **title:** Tytuł artykułu, który zawiera główny nagłówek publikacji.
- **text:** Pełna treść artykułu, obejmująca cały tekst publikacji.
- **subject:** Kategoria tematyczna artykułu, polityka, gospodarka, co pozwala na analizę kontekstu tematycznego.
- **date:** Data publikacji artykułu, umożliwiająca analizę temporalną i chronologiczną.

Taka struktura danych pozwala na kompleksową analizę artykułów pod kątem ich treści, kontekstu tematycznego oraz ram czasowych.

Liczba próbek

Zbiór zawiera około **40 000 artykułów**, podzielonych równomiernie między kategorie "Fake" i "True", co zapewnia zrównoważony zestaw danych do trenowania i testowania modeli.

Cechy tekstowe

Próbki zostały przetworzone w formie wektorów cech, co pozwala na uchwycenie znaczenia słów w kontekście dokumenty.

2.8.2. WELFake Dataset

Zbiór danych WELFake został stworzony w celu analizowania i wykrywania fałszywych wiadomości, zwłaszcza w kontekście mediów społecznościowych, gdzie szybkie rozprzestrzenianie się informacji ułatwia rozprzestrzenianie się fałszywych informacji. Zostało to szczegółowo opisane w pracy [21], w której autorzy zaproponowali model WELFake bazujący na osadzeniach słów i cechach językowych, służący do klasyfikowania fałszywych wiadomości za pomocą uczenia maszynowego. Zbiór danych WELFake powstał w wyniku połączenia czterech istniejących zbiorów danych: Kaggle Fake News, McIntire, Reuters i BuzzFeed. Każda z tych kolekcji zawiera różnorodne źródła danych, co sprawia, że WELFake jest szczególnie cenny w badaniach nad dezinformacją. Na przykład dane Kaggle obejmują szeroki zakres fałszywych i prawdziwych artykułów, podczas gdy zbiór danych BuzzFeed dostarcza dane specyficzne dla mediów społecznościowych, które umożliwiają analizę różnych kontekstów i stylów tekstu.

Struktura zbioru danych

Zbiór WELFake składa się z dwóch głównych kategorii:

- **Fake News:** Artykuły fałszywe, często o charakterze manipulacyjnym, pochodzące z różnych niesprawdzonych źródeł, takich jak strony internetowe znane z publikowania clickbaitów lub treści dezinformujących.
- **True News:** Artykuły prawdziwe, oparte na wiarygodnych źródłach, takich jak renomowane portale informacyjne, które stosują standardy dziennikarskie.

Przetwarzanie danych

Dane w zbiorze WELFake wymagały dodatkowego przetworzenia, aby dostosować je do naszych potrzeb. Struktura tekstu została zmodyfikowana w następujący sposób: usunięto nadmiarowe spacje, problematyczne symbole (znaki specjalne, które mogłyby zakłócać przetwarzanie tekstu), a także ustalono jednolite kodowanie UTF-8 dla wszystkich próbek. Ostateczna struktura danych została zdefiniowana w następujący sposób:

- `title`: Tytuł artykułu, pochodzący z oryginalnego pola `row['title']`.

- `text`: Treść artykułu, nabyta z pola `row['text']`.
- `subject`: Kategoria tematyczna, ustawiona na stałą wartość `'news'`.
- `date`: Data publikacji.

Liczba próbek

Zbiór WELFake zawiera dokładnie **72 134 artykuły**, z podziałem na kategorie fałszywych i prawdziwych wiadomości, jak podano w [21]. Ta duża liczba próbek czyni WELFake jednym z największych zbiorów danych użytych w naszych eksperymentach, co pozwala na skuteczne trenowanie i testowanie modeli.

2.8.3. BuzzFeed News Dataset

BuzzFeed News Dataset został stworzony w 2016 roku przez zespół BuzzFeed News w celu analizy wpływu fałszywych i prawdziwych wiadomości na zaangażowanie użytkowników w mediach społecznościowych, w szczególności w kontekście wyborów prezydenckich w USA w 2016 roku. Szczegółowy opis jest w artykule [18], w którym autorzy przeanalizowali, jak hiperpartyjne i fałszywe wiadomości rozprzestrzeniały się na platformie Facebook w kluczowych miesiącach przed wyborami. Dane zostały zebrane z dziewięciu stron na Facebooku, należących do trzech kategorii: popularnych mediów (CNN, The New York Times), stron lewicowych (Addicting Info) oraz stron prawicowych (Eagle Rising). Dodatkowo, autorzy zidentyfikowali strony znane z publikowania fałszywych wiadomości, takie jak Ending the Fed, które były odpowiedzialne za znaczną część najbardziej angażujących treści w okresie wyborczym. Zbiór ten zawiera 2 282 posty z tych stron, opublikowane między 20 a 27 września 2016 roku, które następnie zostały powiązane z pełnymi artykułami, jeśli linki były dostępne.

Badanie [18] wykazało, że fałszywe wiadomości generowały znacznie większe zaangażowanie na Facebooku w porównaniu z treściami popularnymi, co podkreśla istotność analizy takich danych w kontekście zrozumienia dynamiki dezinformacji w mediach społecznościowych. Zbiór BuzzFeed News Dataset jest szczególnie wartościowy, ponieważ koncentruje się na zaangażowaniu użytkowników, co pozwala na analizę nie tylko treści artykułów, ale także ich wpływu społecznego. W naszym badaniu wykorzystaliśmy ten zbiór do analizy różnic między fałszywymi a prawdziwymi wiadomościami pod kątem ich cech tekstowych i wpływu na odbiorców, co czyni go cennym uzupełnieniem dla większych zbiorów, takich jak ISOT czy WELFake.

Struktura zbioru danych

Zbiór składa się z dwóch głównych kategorii:

- **Fake News:** Artykuły oznaczone jako fałszywe, często pochodzące z sensacyjnych stron internetowych, takich jak Ending the Fed. Charakteryzują się chwytliwymi nagłówkami, emocjonalnym językiem i brakiem wiarygodnych źródeł.
- **True News:** Artykuły prawdziwe, publikowane przez popularne media, takie jak The New York Times czy CNN, które stosują wysokie standardy dziennikarskie.

Przetwarzanie danych

Dane w zbiorze BuzzFeed zostały przetworzone w celu ujednolicenia ich struktury. Stworzono nową ramkę danych (`new_df`) w następujący sposób:

- `title`: Tytuł artykułu, pochodzący z oryginalnego pola `df['title']`.
- `text`: Treść artykułu, nabyta z pola `df['text']`.
- `subject`: Informacje o kategorii tematycznej, pobrane z pola `df['meta_data']`, które zawierało dodatkowe metadane, takie jak informacje o źródle publikacji.
- `date`: Data publikacji, nabyta z pola `df['publish_date']`.

Taki format pozwolił na łatwe wykorzystanie danych w dalszych eksperymentach, zachowując kluczowe informacje o artykułach.

Liczba próbek

Zbiór BuzzFeed zawiera **2 282 artykuły**, z podziałem na kategorie fałszywych i prawdziwych wiadomości, co zgadza się z liczbą postów podaną w [18]. Choć jest to zbiór mniejszy niż ISOT czy WELFake, jego specyfika (skupienie na mediach społecznościowych i zaangażowaniu użytkowników) czyni go wartościowym w kontekście naszych badań.

2.8.4. Porównanie zbiorów danych

Poniższa tabela przedstawia porównanie trzech analizowanych zbiorów danych pod kątem ich kluczowych cech:

Cecha	BuzzFeed	ISOT	WELFake
Liczba próbek	~2 282	~40 000	~72 134
Źródła danych	BuzzFeed News, Facebook	University of Victoria	Kaggle, McIntire, Reuters, BuzzFeed
Główny cel	Analiza wiadomości w mediach społecznościowych	Klasyfikacja fałszywych wiadomości	Wykrywanie fałszywych informacji
Wpływ na wyniki	Dobre wyniki w analizie zaangażowania i emocjonalnego języka, ale ograniczona generalizacja z powodu małego rozmiaru.	Wysoka dokładność w klasyfikacji dzięki dużej liczbie próbek i równowadze klas; dobrze wspiera modele głębokiego uczenia.	Bardzo wysoka dokładność dzięki różnorodności źródeł i dużej liczbie próbek; idealny dla modeli opartych na osadzeniach słów.
Ograniczenia	Mały rozmiar i specyfika mediów społecznościowych ograniczają uniwersalność; brak aktualizacji danych.	Ograniczone do starszych danych, co może wpływać na skuteczność w nowych kontekstach.	Wymaga intensywnego przetwarzania wstępnego; niektóre źródła mogą wprowadzać szum.

Tabela 2.2: Porównanie zbiorów danych: BuzzFeed, ISOT i WELFake pod kątem wpływu na wyniki.

2.9. IMPLEMENTACJA ANALIZY WYNIKÓW

W podanym rozdziale opisujemy końcowy etap implementacji, wspólny dla wszystkich implementowanych rozwiązań metod uczenia zespołowego, obejmujący ewaluację modeli oraz analizę i wizualizację uzyskanych wyników. Ten proces został zaimplementowany w dwóch głównych etapach i koncentruje się na porównaniu skuteczności klasyfikacji dla metod naukowych, jak i autorskich. Proces ten wykorzystuje dane z kluczowych zestawów danych do metryk wydajności oraz do krzywych precyzji-długości, organizując wyniki według kategorii zestawów danych dla standaryzowanej oceny.

Dane wejściowe do analizy wyników są generowane przez każdą z 20 metod (metoda1 do metoda20), które zwracają wytrenowany model oraz dane testowe w następującej postaci:

- **model** - wytrenowany klasyfikator, reprezentujący implementację danej metody, gotowy do generowania predykcji.

- **X_test** - tablica zawierająca cechy zbioru testowego, które mogą być wektorami TF-IDF lub osadzeniami kontekstowymi z BERT, RoBERTa czy Sentence Transformers, wykorzystywanymi do oceny modelu.
- **y_test** - tablica zawierająca etykiety binarne zbioru testowego: 0 dla fałszywych wiadomości, 1 dla prawdziwych; służące jako punkt odniesienia dla predykcji.

Te dane są przekazywane do dalszej ewaluacji i analizy, a ich spójna struktura umożliwia standaryzowane porównanie wyników.

Pierwszym krokiem jest ewaluacja modeli przy użyciu funkcji `evaluate_model`, wywoływanej dla każdej metody po jej uruchomieniu. Poniżej znajduje się szczegółowy opis parametrów przekazywanych do funkcji `evaluate_model`:

- **model** - zwrócony parametr wyniku metody, szczegółowo opisaliśmy poprzednio.
- **X_test** - zwrócony parametr wyniku metody, szczegółowo opisaliśmy poprzednio.
- **y_test** - zwrócony parametr wyniku metody, szczegółowo opisaliśmy poprzednio.
- **run_type** - identyfikator typu uruchomienia, używany do oznaczania wyników w plikach wyjściowych.
- **output_path** - ścieżka do katalogu, w którym zapisywane są wyniki, umożliwiającą organizację danych wyjściowych.
- **start_time** - znacznik czasu rozpoczęcia uruchomienia, służący do obliczania całkowitego czasu wykonania.

Wspominając o szczegółach tego kroku, funkcja rozpoczyna od przeprowadzenia walidacji krzyżowej, obliczając średnią dokładność (`cv_accuracy_mean`) oraz odchylenie standardowe (`cv_accuracy_std`). Walidacja krzyżowa to metoda oceny modelu, w której dane testowe dzieli się na 5 równych części - model trenuje się na 4 z nich, a testuje na jednej, powtarzając proces 5 razy, za każdym razem zmieniając testowaną część. Dzięki temu można ocenić, jak stabilny i niezawodny jest model w różnych warunkach, zamiast sprawdzać go tylko na jednym zestawie danych. Następnie model generuje predykcje prawdopodobieństw - dla standardowych struktur danych za pomocą `predict_proba`, a dla zaawansowanych struktur, takich jak sieci neuronowe, za pomocą `predict`. Te prawdopodobieństwa są konwertowane na etykiety binarne przy progu 0.5, gdzie wartość powyżej progu oznacza klasę pozytywną (1). Obliczane są metryki oceny, w tym:

- **Dokładność (Accuracy)** - ile przewidywań było poprawnych.
- **Precyzja (Precision)** - jak często, gdy model mówi "prawda", ma rację.
- **Recall** - jak dobrze znajduje prawdziwe wiadomości.
- **Wynik F1 (F1-Score)** - równowaga między precyzją a czułością.
- **Pole pod krzywą ROC-AUC (ROC-AUC)** - jak dobrze model odróżnia klasy.
- **Współczynnik korelacji Matthews (MCC)** - miara zgodności przewidywań z rzeczywistością.

- **Logarytmiczna strata (Log Loss)** - jak bardzo model się myli.
- **Współczynnik Kappa Cohena (Cohen's Kappa)** - jak dobrze model przewyższa losowe zgadywanie.

Dodatkowo mierzony jest czas wykonania (`Execution Time`), a wyniki walidacji krzyżowej są dołączane do metryk. Wszystkie wartości są zapisywane do pliku CSV w formacie `runX_results.csv`, a krzywa Precision-Recall (`precision_recall_curve`) jest generowana i zapisywana jako `runX_pr_curve.csv`. Na koniec metryki są wyświetlane w konsoli, co pozwala na szybką weryfikację skuteczności modelu. W tym procesie model służy do generowania predykcji, `X_test` i `y_test` wspierają obliczanie metryk i walidację krzyżową, `run_type` oraz `output_path` określają format i lokalizację zapisu wyników, a `start_time` pozwala zmierzyć czas wykonania. Kluczowe parametry wykorzystane w procesie ewaluacji modeli obejmują:

- **`n_splits=5`** - liczba podziałów w walidacji krzyżowej, określająca, ile razy dane są dzielone na zestawy treningowe i testowe w celu obliczenia średniej dokładności.
- **`random_state=42`** - ziarno losowości, zapewniające odtwarzalność podziałów danych w walidacji krzyżowej i podziale treningowym/testowym.
- **`threshold=0.5`** - próg prawdopodobieństwa, powyżej którego predykcje są klasyfikowane jako klasa pozytywna (1), stosowany w konwersji prawdopodobieństw na etykiety binarne.
- **`zero_division=0`** - wartość domyślna dla obsługi dzielenia przez zero w metrykach precyzji i recall, zapobiegająca błędom przy braku predykcji jednej z klas.
- **`flatten=True`** - parametr określający, czy predykcje z modeli Keras powinny być spłaszczone do jednowymiarowej tablicy, ułatwiając dalsze przetwarzanie.

Drugim krokiem jest analiza i wizualizacja wyników, realizowana przez skrypt generujący wykresy porównawcze. Skrypt ten przetwarza wyniki z folderów z wynikami testów, obejmujących uruchomienia metod od 1 do 20 oraz dodatkowe pliki porównawcze. Wyniki ze wszystkich plików są łączone w jedną tabelę danych z dodaną kolumną Metoda wskazującą numer metody, których posiadamy 20. Dla metryki `Execution Time (s)` usuwane są wartości odstające. Następnie generowane są wykresy słupkowe dla wszystkich wspomnianych metryk dla danych metod autorskich oznaczonych na czerwono i naukowych na niebiesko. Dodatkowo, dla wszystkich uruchomień generowany jest wykres krzywych Precision-Recall na podstawie danych z plików `runX_pr_curve.csv`.

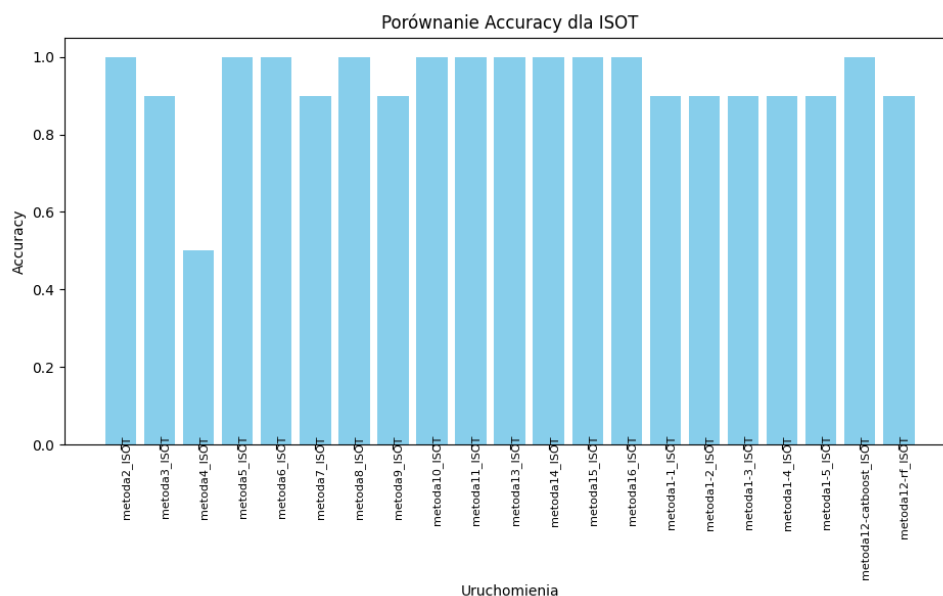
2.10. WSTĘPNA ANALIZA WYNIKÓW

W niniejszym podrozdziale przeprowadzamy wstępną analizę wyników, koncentrując się wyłącznie na zbiorze danych ISOT Fake News Dataset, który posłuży jako baza do przykładów inspirujących nasze autorskie metody. Szczegółowe omówienie wyników

na tych danych umożliwi nam wypracowanie i implementację innowacyjnych podejść, otwierając drogę do przełomowych rozwiązań. Skupiamy się na dokładnym opisie wyników z wykorzystaniem paradygmatów few-shot learning oraz Sentence-BERT (SBERT). Wybór few-shot wynika z jego zdolności do efektywnego uczenia się na małej liczbie przykładów, co jest kluczowe w dynamicznie zmieniającym się środowisku dezinformacji, w przeciwieństwie do no-shot czy one-shot, które nie zapewniają odpowiedniej elastyczności. Z kolei SBERT został wybrany ze względu na jego zaawansowane możliwości semantycznego modelowania tekstu, przewyższające tradycyjne modele jak BERT czy RoBERTa, szczególnie w zadaniach wymagających analizy kontekstowej i porównawczej.

2.10.1. Dokładność (Accuracy)

W tej analizie oceniamy dokładność (Accuracy), której proponujemy się przejrzeć na rysunku 2.5. Najwyższe wyniki, osiągające wartość 1.0, zanotowały metody **metoda2**, **metoda5**, **metoda6**, **metoda8**, **metoda9**, **metoda10**, **metoda11**, **metoda13**, **metoda14**, **metoda15**, **metoda16**, **metoda12_catboost**, co dowodzi ich wyjątkowej skuteczności w zadaniu klasyfikacji. Z kolei najgorzej wypadła **metoda4**, uzyskując wyniki w przedziale 0.4–0.6, co wskazuje na jej niską efektywność i konieczność dalszej poprawy. Pozostałe rozwiązania prezentują się dość przeciętnie. Te wnioski motywują nas do projektowania nowatorskich podejść, które będą czerpały z metod osiągających najlepszą dokładność.

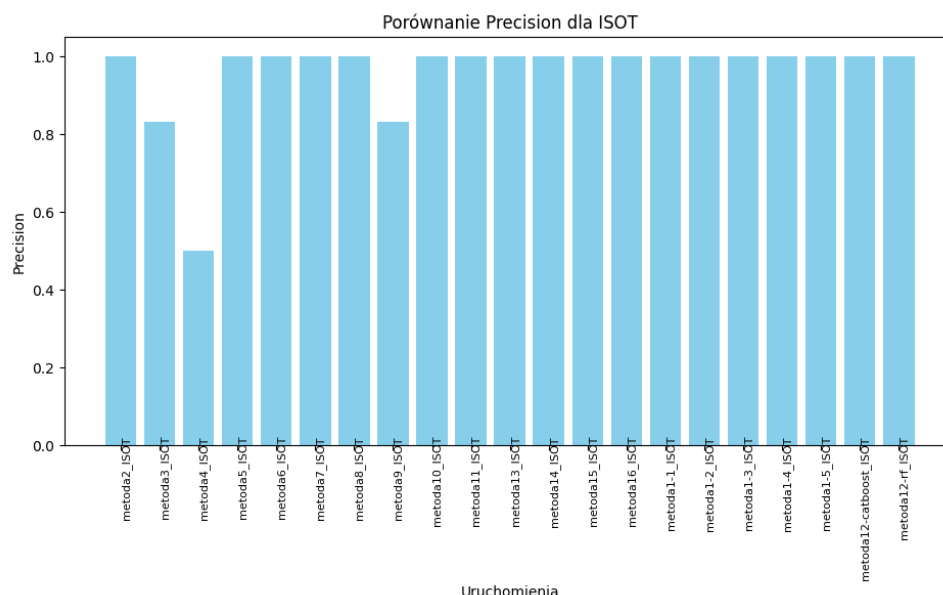


Rys. 2.5: Porównanie Accuracy dla ISOT Fake News Dataset dla różnych uruchomień metod.

2.10.2. Precyzja (Precision)

W tym podrozdziale przyglądamy się precyzji (Precision), co zobrazowano na rysunku 2.6. Najslabiej wypadła **metoda4**, z wynikiem w przedziale 0.4–0.6, co wskazuje na jej

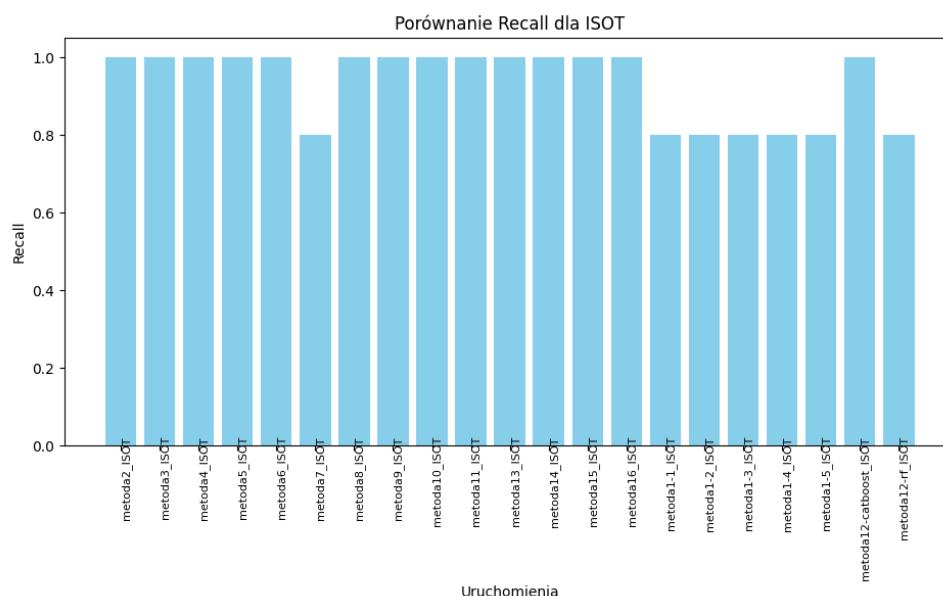
niską efektywność i potrzebę dalszego doskonalenia. **metoda3** oraz **metoda9** prezentują umiarkowaną wydajność. Natomiast najwyższe wyniki osiągnęły pozostałe metody, co świadczy o ich wyjątkowej skuteczności w identyfikacji prawdziwych przypadków. Te obserwacje motywują do bardziej rozbudowanego opracowania wyników według kolejnych kryteriów, ponieważ nie udało nam się ograniczyć liczby potencjalnych metod, które pozwolą nam zaprezentować autorskie rozwiązania.



Rys. 2.6: Porównanie Precision dla ISOT dla różnych uruchomień metod.

2.10.3. Recall

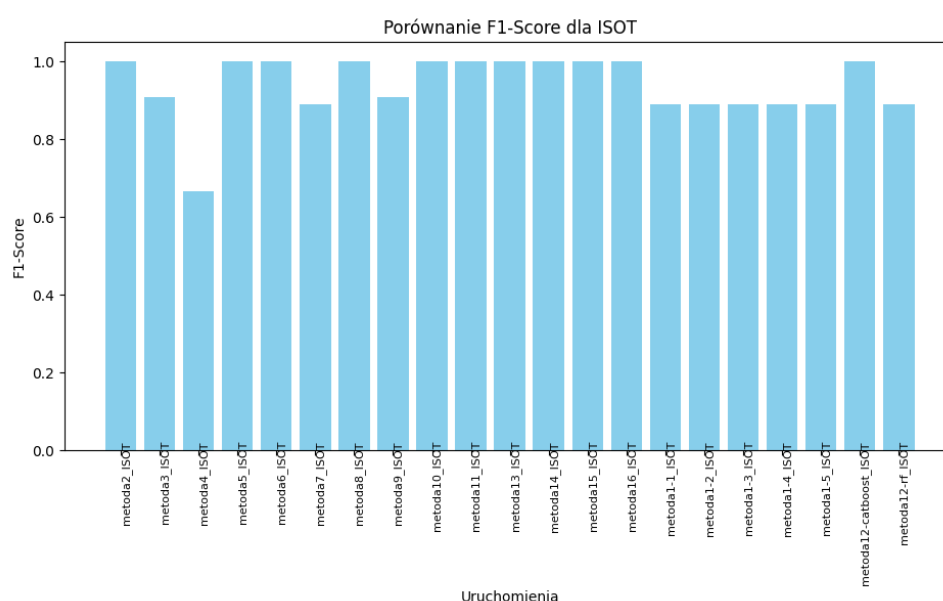
W tej części badamy Recall, co ukazano na rysunku 2.7. Najwyższe rezultaty, sięgające wartości 1.0, osiągnęły metody **metoda2**, **metoda3**, **metoda4**, **metoda5**, **metoda6**, **metoda8**, **metoda9**, **metoda10**, **metoda11**, **metoda12**, **metoda13**, **metoda14**, **metoda15**, **metoda16** oraz **metoda12_catboost**, co podkreśla ich wybitną skuteczność w wykrywaniu istotnych przypadków. Pozostałe podejścia osiągają przeciętne rezultaty. Warto podkreślić, że wyniki danych metod dla Recall pokrywają się z rezultatami w porównaniu dokładności (Accuracy), co ułatwia dalszy wybór inspiracji oraz modeli dla autorskich metod.



Rys. 2.7: Porównanie Recall dla ISOT dla różnych uruchomień metod.

2.10.4. Wynik F1 (F1-Score)

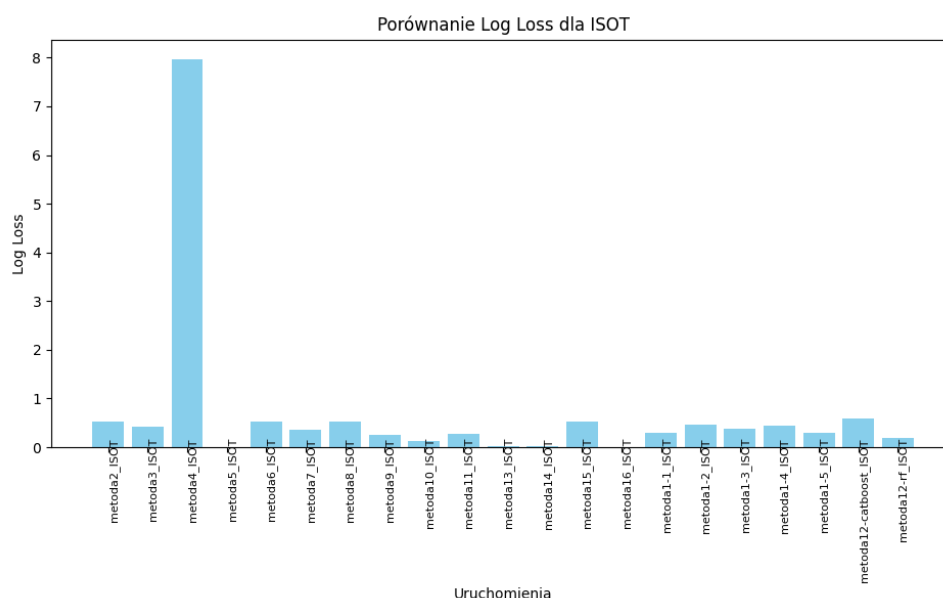
W tej sekcji przyglądamy się wynikowi F1 (F1-Score), któremu proponujemy się przejrzeć na rysunku 2.8. Najlepsze rezultaty, osiągające wartość 1.0, uzyskały metody **metoda2**, **metoda5**, **metoda6**, **metoda7**, **metoda8**, **metoda9**, **metoda10**, **metoda11**, **metoda12**, **metoda13**, **metoda14**, **metoda15**, **metoda16** oraz **metoda12_catboost**, co po raz kolejny potwierdza ich wybitną skuteczność. Pozostałe podejścia osiągają średni poziom wydajności, co już pozwala nam ich ograniczyć, patrząc na zachowanie podobnej tendencji dla wyników.



Rys. 2.8: Porównanie F1-Score dla ISOT dla różnych uruchomień metod.

2.10.5. Logarytmiczna strata (Log-Loss)

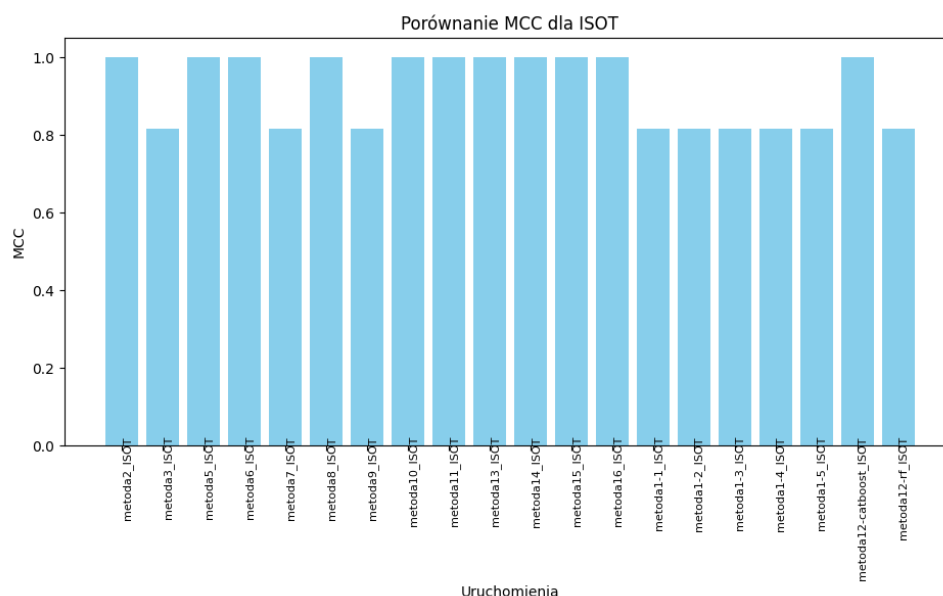
W tej części oceniamy logarytmiczną stratę (Log-Loss), co zobrazowano na rysunku 2.9. W przypadku tej metryki najlepsze wyniki, czyli najniższe wartości bliskie 0.0, osiągnęły metody **metoda5**, **metoda10**, **metoda13**, **metoda14**, **metoda16** oraz **metoda12_catboost**, co świadczy o ich wysokiej skuteczności w minimalizowaniu błędu predykcji. Pozostałe podejścia prezentują się na przeciętnym poziomie. Te obserwacje inspirują do zwrócenia uwagi oraz szczególnego porównania dla metod o najlepszych wynikach, ponieważ dokładnie one się wybijają na tle innych.



Rys. 2.9: Porównanie Log-Loss dla ISOT dla różnych uruchomień metod.

2.10.6. Współczynnik korelacji Matthews (MCC)

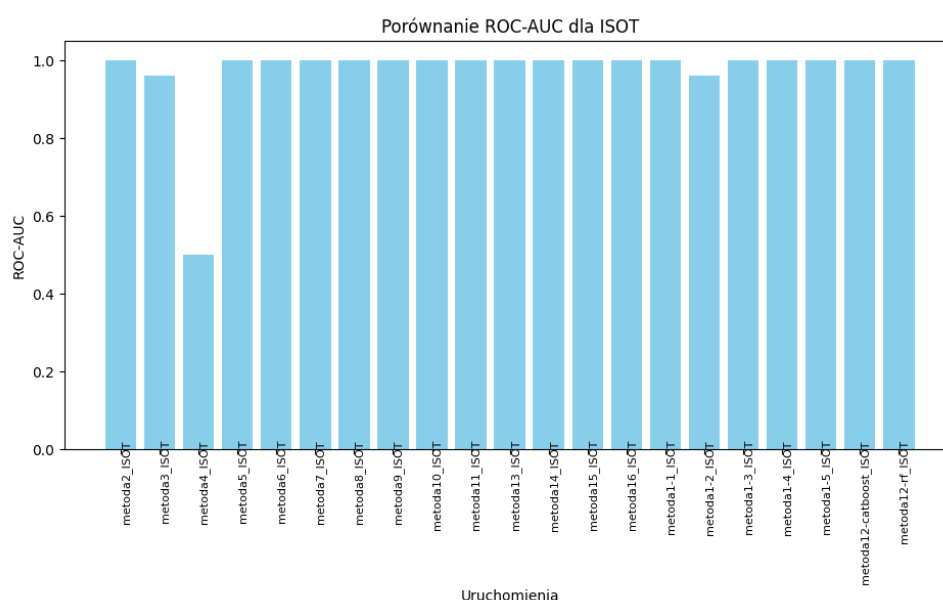
W tej części przyglądamy się współczynnikowi korelacji Matthews (MCC), co przedstawiono na rysunku 2.10. Najlepsze wyniki, sięgające wartości 1.0, osiągnęły metody **metoda5**, **metoda10**, **metoda13**, **metoda14**, **metoda16** oraz **metoda12_catboost**. Pozostałe podejścia osiągają zróżnicowane wyniki, co nie wpływa na nasze wnioski, ponieważ skupiliśmy się na porównaniu faworytów według poprzednich kryteriów. Te wyniki wskazują na użyteczność tego kryterium w ocenie metod, co motywuje nas do skupienia się na najlepszych rozwiązaniach w dalszej analizie.



Rys. 2.10: Porównanie MCC dla ISOT dla różnych uruchomień metod.

2.10.7. Pole pod krzywą ROC-AUC (ROC-AUC)

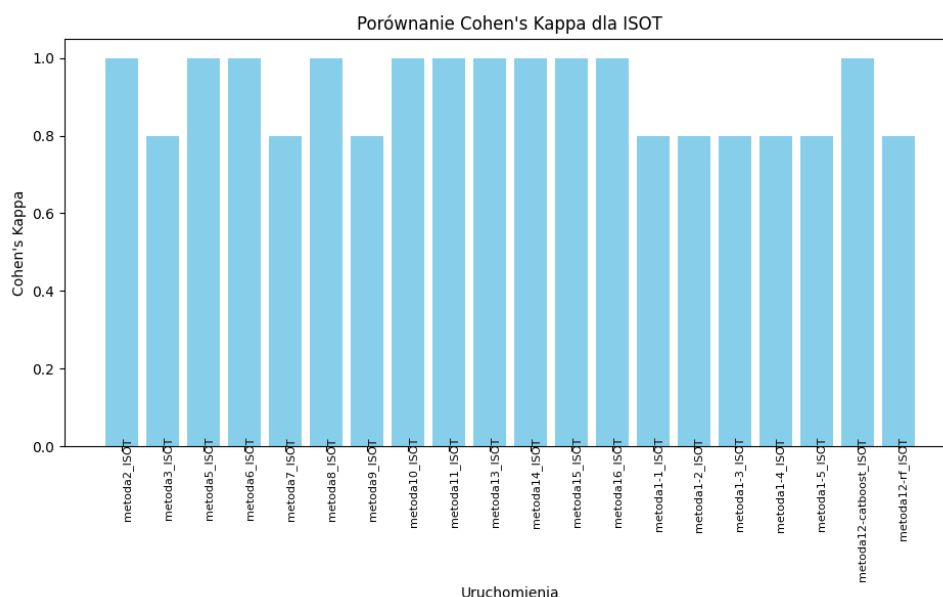
W tej sekcji analizujemy pole pod krzywą ROC-AUC (ROC-AUC), co zilustrowano na rysunku 2.11. Najwyższe wartości, osiągające 1.0, zanotowały praktycznie wszystkie metody, oprócz **metoda3**, **metoda4** oraz **metoda1-2**. Ciężko tutaj wyróżnić faworytów lub metody, które szczególnie się wybijają, dlatego warto będzie ich porównać również pod innymi względami.



Rys. 2.11: Porównanie ROC-AUC dla ISOT dla różnych uruchomień metod.

2.10.8. Współczynnik Kappa Cohena (Cohen's Kappa)

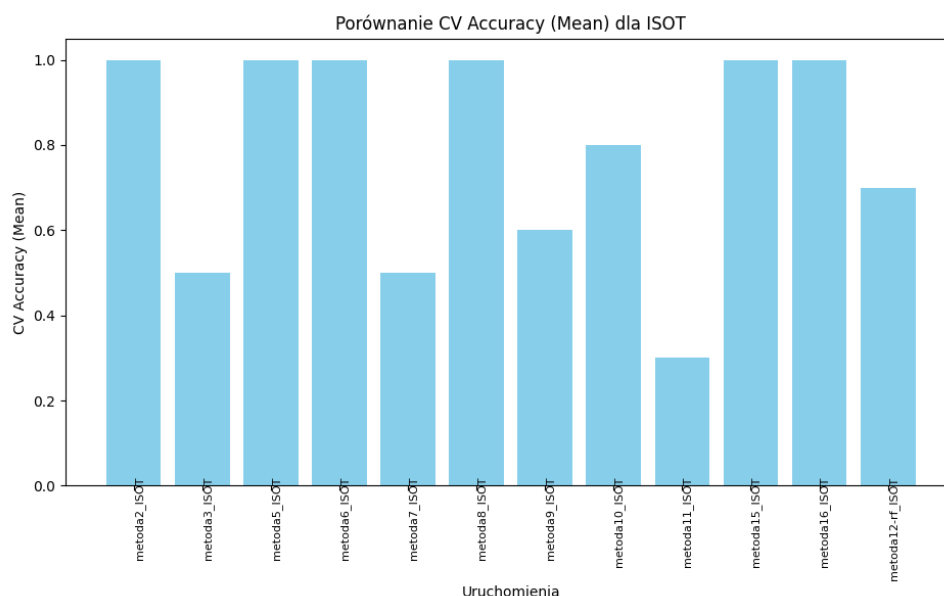
W tej części oceniamy współczynnik Kappa Cohena (Cohen's Kappa), co przedstawiono na rysunku 2.12. Najwyższe wartości, zbliżone do 1.0, osiągnęły metody **metoda2**, **metoda5**, **metoda6**, **metoda8**, **metoda9**, **metoda10**, **metoda11**, **metoda12**, **metoda13**, **metoda14**, **metoda15**, **metoda16** oraz **metoda12_catboost**, co świadczy o ich doskonałej zgodności przewidywań z rzeczywistymi wynikami. Natomiast inne metody wypadły z wynikiem poniżej 0.9, co wskazuje na ich umiarkowane rezultaty.



Rys. 2.12: Porównanie Cohen's Kappa dla ISOT dla różnych uruchomień metod.

2.10.9. Średnia (Mean)

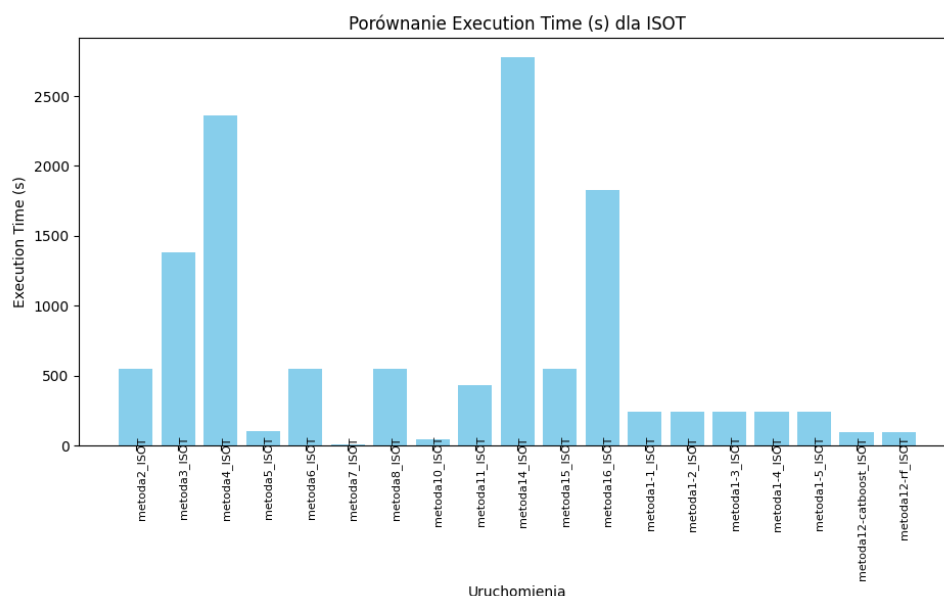
W niniejszym podrozdziale przyglądamy się średniej (Mean), co ukazano na rysunku 2.13. Najwyższe wartości, oscylujące wokół 1.0, osiągnęły metody **metoda2**, **metoda5**, **metoda6**, **metoda8**, **metoda15** oraz **metoda16**. Najslabiej wypadła **metoda11**, z wynikiem poniżej 0.3, co wskazuje na jej ograniczoną efektywność oraz sugeruje, że wspomniana metoda nie nadaje się do czerpania inspiracji ani pomysłów na tworzenie autorskich rozwiązań. Inne metody osiągnęły średnie wyniki, dlatego proponuję nie skupiać się na nich zbyt wiele na obecnym etapie.



Rys. 2.13: Porównanie Mean dla ISOT dla różnych uruchomień metod.

2.10.10. Czas wykonania (Execution Time)

W tej sekcji analizujemy czas wykonania (Execution Time), któremu proponujemy się przejrzeć na rysunku 2.14. Najlepsze wyniki, czyli najkrótszy czas wykonania wynoszący około 0.1–0.2 sekundy, osiągnęły metody **metoda5**, **metoda7**, **metoda10** oraz **metoda12_catboost**, co świadczy o ich wysokiej efektywności obliczeniowej. Najślabsze wyniki osiągnęły **metoda4**, **metoda14** oraz **metoda16**, z czasem wykonania przekraczającym 1200 sekund. Pozostałe rozwiązania prezentują się dość przeciętnie. Te wyniki podkreślają znaczenie tego kryterium w doborze efektywnych rozwiązań, co pozwala na tworzenie autorskich metod o lepszej wydajności obliczeniowej i krótszym czasie wykonania.



Rys. 2.14: Porównanie Execution Time dla ISOT dla różnych uruchomień metod.

2.10.11. Podsumowanie wstępnej analizy wyników metod

W tej części wywnioskujemy, czym warto się inspirować tworząc rozwiązania autor-skie, dzięki wynikom dla różnych metod na zbiorze danych ISOT Fake News Dataset, patrząc do tabeli 2.3. Tabela ukazuje, że metody takie jak **metoda5**, **metoda10** oraz **metoda12_catboost** osiągają najlepsze wyniki w większości kryteriów, w tym w Dokładności, F1-Score, MCC oraz Cohen's Kappa, uzyskując wartości bliskie 1.0. Szczególną uwagę zwraca **metoda5**, która wyróżnia się również w Log-Loss i Czasie wykonania, co czyni ją potencjalnym kandydatem do dalszego rozwoju. Z kolei metody takie jak **metoda3** i **metoda4** wykazują najsłabsze rezultaty, szczególnie w Precyzji i Średniej, co sugeruje ich ograniczoną użyteczność w tym kontekście.

Metoda	Dokładność	Precyzja	Recall	F1-Score	Log-Loss	MCC	ROC-AUC	Cohen's Kappa	Średnia	Czas wykonania	Najlepsze dopasowanie
metoda2	x		x	x			x	x	x		
metoda3			x								
metoda4			x								
metoda5	x	x	x	x	x	x	x	x	x	x	x
metoda6	x		x	x			x	x	x		
metoda7			x	x			x			x	
metoda8	x	x	x	x			x	x	x		
metoda9	x	x	x	x			x	x			
metoda10	x		x	x	x	x	x	x		x	x
metoda11	x		x	x			x	x			
metoda12_rf			x	x			x	x			
metoda12_catboost	x		x	x	x	x	x	x		x	x
metoda1-1							x				
metoda1-2											
metoda1-3							x				
metoda1-4							x				
metoda1-5							x				

Tabela 2.3: Porównanie metod według kryteriów i najlepsze dopasowanie.

3. WŁASNE METODY

Autorskie metody powstały w oparciu o szkielet oraz koncepcje metod, które w badaniach wykazały najlepsze rezultaty. Krótko się przejrzałyśmy wynikom w 2.10. Wykorzystując wnioski płynące z analizy ich wzorców, opracowaliśmy nowatorskie podejścia, które skutecznie łączą precyzję z wydajnością. Rozwiązania te zostały starannie dostosowane do wymagań i złożoności problematyki wykrywania **fake news**, gwarantując wysoką jakość i efektywność.

3.1. PIERWSZA METODA (METODA17)

3.1.1. Pomysł na metodę

Metoda17 jest modelem zespołowym, który łączy predykcje trzech różnych klasyfikatorów: wielowarstwowego perceptronu (`MLPClassifier`), regresji logistycznej (`LogisticRegression`) i XGBoosta (`XGBClassifier`) za pomocą miękkiego głosowania. Inspiracją dla tej metody jest **Metoda10**, która również wykorzystuje `VotingClassifier` do łączenia różnych modeli. Kluczowym ulepszeniem w **Metoda17** jest zastosowanie optymalizacji hiperparametrów (przynajmniej dla regresji logistycznej) oraz skalowania cech dla każdego z bazowych klasyfikatorów przed ich połączeniem. Celem jest wykorzystanie komplementarnych mocnych stron różnych algorytmów i poprawa ogólnej wydajności predykcyjnej poprzez dostrojenie parametrów i ujednolicenie skali danych wejściowych.

3.1.2. Wejściowe parametry

Metoda `metoda17` przyjmuje następujące parametry wejściowe:

1. **X_train**: Macierz danych treningowych dla n próbek, m cech. Każdy wiersz reprezentuje próbkę, a każda kolumna cechę. Wartości w macierzy są zazwyczaj liczbowe.
2. **y_train**: Wektor etykiet dla danych treningowych n próbek. Każdy element odpowiada próbce w **X_train** i zawiera jej prawdziwą klasę, zazwyczaj 0 i 1 dla klasyfikacji binarnej, liczby całkowite dla wieloklasowej.
3. **X_test**: Macierz danych testowych. Struktura podobna do **X_train**, używana do oceny modelu po treningu.
4. **y_test**: Wektor etykiet dla danych testowych. Prawdziwe klasy dla próbek w **X_test**, używane do oceny wyników modelu.

3.1.3. Wyjściowe parametry

Metoda zwraca następujące elementy:

1. **voting_clf_final**: Wytrenowany model `VotingClassifier` po optymalizacji.
2. **X_test_scaled**: Przeskalowane cechy zbioru testowego.
3. **y_test**: Wektor etykiet testowych.

3.1.4. Algorytm

1. Przygotowanie i trenowanie modeli bazowych:
 - **Wielowarstwowy Perceptron (MLPClassifier)**: Inicjalizuje się model MLP. Tworzy się potok (Pipeline) ze skalowaniem cech (`StandardScaler`) i trenuje na **X_train** i **y_train**.
 - **Regresja Logistyczna (LogisticRegression)**: Tworzy się potok (Pipeline) ze skalowaniem cech (`StandardScaler`) i regresją logistyczną. Definiuje się przestrzeń hiperparametrów i przeprowadza optymalizację za pomocą `GridSearchCV` na **X_train** i **y_train**. Wybiera się najlepszy model.
 - **XGBoost (XGBClassifier)**: Inicjalizuje się model XGBoost. Tworzy się potok (Pipeline) ze skalowaniem cech (`StandardScaler`) i trenuje na **X_train** i **y_train**.
2. Stworzenie i trenowanie modelu głosującego:
 - Łączy się wytrenowane modele bazowe (MLP, zoptymalizowana regresja logistyczna, XGBoost) w model zespołowy (`VotingClassifier`) z miękkim głosowaniem.
 - Trenuje się model głosujący na **X_train** i **y_train**.
3. Przeskalowanie danych testowych:
 - Używa się skalerów wytrenowanych na **X_train** do przeskalowania **X_test** dla każdego z modeli bazowych (osobno dla regresji logistycznej, MLP i XGBoost, jeśli były w potokach).
 - Ostatecznie, może zostać użyte jedno skalowanie dla całego **X_test** do oceny modelu głosującego.
4. Zwracanie wyników: Zwraca się wytrenowany model głosujący (**voting_clf_final**), przeskalowane cechy zbioru testowego (**X_test_scaled**) oraz oryginalne etykiety testowe (**y_test**).

3.1.5. Kod

```
# Inicjalizacja modeli: MLP, XGBoost, regresja logistyczna
# w potoku ze skalowaniem i optymalizacją hiperparametrów.
mlp = MLPClassifier(...)
xgb = XGBClassifier(...)
pipeline_logreg = Pipeline(
```

```

[('scaler', StandardScaler()),
 ('log_reg', LogisticRegression(...))
])

# Definiowanie przestrzeni hiperparametrów do
# przeszukania dla regresji logistycznej.
param_grid_logreg = {...}

# Wyszukiwanie najlepszych hiperparametrów dla regresji logistycznej.
grid_search_logreg = GridSearchCV(pipeline_logreg, param_grid_logreg, ...)
grid_search_logreg.fit(X_train, y_train)
log_reg_optimized = grid_search_logreg.best_estimator_.named_steps['log_reg']
scaler_logreg = grid_search_logreg.best_estimator_.named_steps['scaler']
X_test_scaled_logreg = scaler_logreg.transform(X_test)

# Potok dla MLP ze skalowaniem.
pipeline_mlp = Pipeline([('scaler', StandardScaler()), ('mlp', mlp)])
pipeline_mlp.fit(X_train, y_train)
X_test_scaled_mlp = pipeline_mlp.named_steps['scaler'].transform(X_test)
mlp_optimized = pipeline_mlp.named_steps['mlp']

# Potok dla XGBoost ze skalowaniem.
pipeline_xgb = Pipeline([('scaler', StandardScaler()), ('xgb', xgb)])
pipeline_xgb.fit(X_train, y_train)
X_test_scaled_xgb = pipeline_xgb.named_steps['scaler'].transform(X_test)
xgb_optimized = pipeline_xgb.named_steps['xgb']

# Stworzenie modelu zespołowego (VotingClassifier) z miękkim głosowaniem.
voting_clf = VotingClassifier(estimators=[
    ('mlp', mlp_optimized),
    ('log_reg', log_reg_optimized),
    ('xgb', xgb_optimized)], voting='soft')

# Trenowanie modelu zespołowego.
voting_clf.fit(X_train, y_train)

# Przeskalowanie danych testowych.
X_test_scaled = StandardScaler().fit_transform(X_test)

```

```
return voting_clf, X_test_scaled, y_test
```

3.1.6. Właściwości analityczne

1. **Efektywność:** Łączenie różnych modeli i optymalizacja hiperparametrów regresji logistycznej ma na celu poprawę dokładności predykcji. Skalowanie cech zapewnia, że wszystkie algorytmy bazowe operują na danych o podobnej skali.
2. **Wykorzystanie różnorodności modeli:** Zastosowanie różnych typów modeli (liniowy, nieliniowy oparty na sieci neuronowej, oparty na drzewach decyzyjnych) pozwala na uwzględnienie różnych wzorców w danych.
3. **Optymalizacja:** Optymalizacja hiperparametrów regresji logistycznej za pomocą GridSearchCV pozwala na znalezienie lepszych ustawień dla tego konkretnego modelu.
4. **Skalowalność:** Metoda może być stosowana do problemów z wieloma cechami i próbkami, jednak czas treningu może być znaczący ze względu na optymalizację hiperparametrów i trening wielu modeli.
5. **Złożoność:** Złożoność obliczeniowa zależy od złożoności każdego z bazowych modeli oraz od procesu optymalizacji hiperparametrów: liczby przeszukiwanych kombinacji parametrów i liczby foldów walidacji krzyżowej.

3.2. DRUGA METODA (METODA18)

3.2.1. Pomysł na metodę

W **Metoda18** integracja **Gradient Boosting** i **CatBoost** została wzbogacona o meta-model w postaci regresji logistycznej. Metoda ta jest wynikiem kombinacji idei pochodzących z **Metoda10** oraz **Metoda13**. W **Metoda10** kluczowym aspektem było miękkie głosowanie, które pozwalało uwzględniać probabilistyczne wyniki modeli bazowych w końcowych predykcjach. Z kolei **Metoda13** została zaimplementowana przy użyciu stacking, gdzie

Gradient Boosting, XGBoost i Random Forest były łączone przy użyciu regresji logistycznej jako meta-modelu.

Metoda18 czerpie z **Metoda13** mechanizm stacking, jednak zamiast wielu modeli bazowych opiera się na dwóch potężnych algorytmach boostingu – **Gradient Boosting** i **CatBoost**. Wybór tych modeli wynika z ich zdolności do radzenia sobie z różnymi typami danych i ich nierównowagą. Regresja logistyczna jako meta-model umożliwia skuteczną integrację wyników, co zostało wcześniej zademonstrowane w **Metoda13**. W porównaniu do **Metoda10**, **Metoda18** idzie krok dalej, stawiając na większą precyzję w meta-modelu poprzez wykorzystanie pełnych rozkładów prawdopodobieństw z boostingu.

3.2.2. Wejściowe parametry

Metoda `train_run18` przyjmuje następujące parametry wejściowe:

1. **X_train**: Macierz danych treningowych, gdzie każdy wiersz to jedna próbka, a kolumny to cechy. Rozmiar n - liczba próbek, m - liczba cech.
2. **y_train**: Wektor etykiet dla danych treningowych. Rozmiar: n , gdzie n - liczba próbek w danych treningowych.
3. **X_test**: Macierz danych testowych, analogiczna do **X_train**, używana do oceny modelu.
4. **y_test**: Wektor etykiet testowych, odpowiadający danym testowym.

3.2.3. Wyjściowe parametry

Metoda zwraca następujące elementy:

1. **meta_model**: Meta-model (regresja logistyczna) wytrenowany na danych pochodzących z predykcji modeli **Gradient Boosting** i **CatBoost**.
2. **meta_features_test**: Macierz meta-cech, zbudowana na podstawie predykcji modeli bazowych dla danych testowych.
3. **y_test**: Wektor etykiet testowych, identyczny z wejściowym **y_test**.

3.2.4. Algorytm

1. Trenowanie modeli bazowych:
 - **Gradient Boosting**: Inicjalizuje się **GradientBoostingClassifier** z **100** estymatorami i **random_state=42**. Następnie przeprowadza się trenowanie na **X_train** i **y_train**.
 - **CatBoost**: Inicjalizuje się **CatBoostClassifier** z **iterations=100**, **random_state=42** i **verbose=0**. Następnie przeprowadza się trenowanie na **X_train** i **y_train**.
2. Generowanie predykcji dla danych treningowych:
 - Oblicza się prawdopodobieństwa klasy pozytywnej $P(y = 1 | x)$: **gb_preds_train** (predykcje **Gradient Boosting**) oraz **cat_preds_train** (predykcje **CatBoost**).
 - Tworzy się macierz meta-cech z **gb_preds** i **cat_preds** jako kolumnami.
3. Trenowanie meta-modelu:
 - Przeprowadza się trenowanie **LogisticRegression** na **meta_features_train** i **y_train**. **LogisticRegression** łączy predykcje modeli bazowych.
4. Generowanie predykcji dla danych testowych:
 - Oblicza się prawdopodobieństwa klasy pozytywnej: **gb_preds_test** (predykcje **Gradient Boosting**) oraz **cat_preds_test** (predykcje **CatBoost**).
 - Tworzy się macierz meta-cech dla danych testowych (analogicznie do treningowych).
5. Zwracanie: Wracany jest wytrenowany **meta_model**, **meta_features_test** i **y_test**.

3.2.5. Kod

```
# Inicjalizacja i trening klasyfikatorow bazowych
gb_clf = GradientBoostingClassifier(
n_estimators=100,
random_state=42
) # Gradient Boosting

cat_clf = CatBoostClassifier(
iterations=100,
verbose=0,
random_state=42
) # CatBoost
gb_clf.fit(X_train, y_train)
cat_clf.fit(X_train, y_train)

# Generowanie predykcji dla danych treningowych
gb_preds_train = gb_clf.predict_proba(X_train)[: , 1]
cat_preds_train = cat_clf.predict_proba(X_train)[: , 1]
meta_features_train = pd.DataFrame({ # Tworzenie meta-cech
'gb_preds': gb_preds_train,
'cat_preds': cat_preds_train
})

# Trenowanie meta-modelu
meta_model = LogisticRegression() # Meta-model
meta_model.fit(meta_features_train, y_train)

# Generowanie predykcji dla danych testowych
gb_preds_test = gb_clf.predict_proba(X_test)[: , 1]
cat_preds_test = cat_clf.predict_proba(X_test)[: , 1]
meta_features_test = pd.DataFrame({ # Meta-cechy dla testu
'gb_preds': gb_preds_test,
'cat_preds': cat_preds_test
})

return meta_model, meta_features_test, y_test
```

3.2.6. Właściwości analityczne

1. Efektywność:

- Kombinacja **Gradient Boosting** i **CatBoost** pozwala na efektywne uchwycenie zarówno liniowych, jak i nieliniowych zależności w danych.
- Meta-model optymalizuje końcowe wyniki poprzez fuzję predykcji z modeli bazowych.

2. Elastyczność: metoda może być zastosowana w różnych problemach klasyfikacyjnych, szczególnie w przypadkach, gdzie różne modele bazowe wykazują różne mocne strony.

3. Stabilność: meta-model działa jako mechanizm konsolidujący, zmniejszając wariancję predykcji i poprawiając stabilność wyników.

4. Złożoność obliczeniowa:

- Zarówno **Gradient Boosting**, jak i **CatBoost** mają złożoność liniową względem liczby próbek i estymatorów, co pozwala na efektywne przetwarzanie dużych zbiorów danych.
- Dodanie meta-modelu zwiększa nieznacznie złożoność, ale poprawia jakość wyników.

5. Wydajność: **CatBoost** jest zoptymalizowany pod kątem szybkiego trenowania, a **Gradient Boosting** zapewnia interpretowalność wyników. Kombinacja tych dwóch modeli z meta-modelem znacząco poprawia skuteczność klasyfikacji.

3.3. TRZECIA METODA (METODA19)

3.3.1. Pomysł na metodę

Metoda19 jest efektem rozwoju pomysłów z **Metoda5**, która opierała się na łączeniu wyników różnych algorytmów. W **Metoda5** zastosowano Random Forest i XGBoost jako modele bazowe, które następnie były integrowane za pomocą regresji logistycznej. **Metoda19** rozwija tę koncepcję, wykorzystując bardziej rozbudowaną architekturę stacking z trzema różnymi modelami bazowymi: Random Forest, XGBoost oraz Gradient Boosting.

W **Metoda19** łączy się Random Forest, oparty na agregacji wielu losowych predyktorów, XGBoost, wykorzystujący technikę gradientowego wzmocnienia, oraz Gradient Boosting, który sekwencyjnie buduje modele korygujące błędy poprzedników. Inspiracją z **Metoda5** jest zastosowanie regresji logistycznej jako meta-modelu, który skutecznie integruje predykcje tych trzech różnorodnych algorytmów. Wykorzystanie wielu, różniących się w swoim działaniu, modeli bazowych ma na celu zwiększenie ogólnej dokładności i stabilności predykcji.

3.3.2. Wejściowe dane

Metoda `train_run19` przyjmuje następujące parametry wejściowe:

1. **X_train**: Macierz danych treningowych dla n próbek, m cech. Każdy wiersz reprezentuje próbkę, a każda kolumna cechę. Wartości w macierzy są zazwyczaj liczbowe.
2. **y_train**: Wektor etykiet dla danych treningowych. Rozmiar: n , gdzie n - liczba próbek w danych treningowych.
3. **X_test**: Macierz danych testowych, analogiczna do **X_train**, używana do oceny modelu.
4. **y_test**: Wektor etykiet testowych, odpowiadający danym testowym.

3.3.3. Wyjściowe dane

Metoda zwraca następujące elementy:

1. **meta_model**: Meta-model (regresja logistyczna) wytrenowany na danych pochodzących z predykcji modeli **Random Forest**, **XGBoost** i **Gradient Boosting**.
2. **meta_features_test**: Macierz meta-cech, zbudowana na podstawie predykcji modeli bazowych dla danych testowych.
3. **y_test**: Wektor etykiet testowych, identyczny z wejściowym **y_test**.

3.3.4. Algorytm

1. Skalowanie danych: Przeprowadza się przeskalowanie **X_train** i **X_test** za pomocą **StandardScaler**.
2. Inicjalizacja i optymalizacja modeli bazowych:
 - Inicjalizuje się **RandomForestClassifier** z **n_estimators=200**, **max_depth=10** i **random_state=42**.
 - Inicjalizuje się **XGBClassifier** z **n_estimators=200**, **use_label_encoder=False**, **eval_metric='logloss'**, **learning_rate=0.05**, **max_depth=5** i **random_state=42**.
 - Inicjalizuje się **GradientBoostingClassifier** z **n_estimators=100**, **learning_rate=0.1**, **max_depth=3** i **random_state=42**.
3. Generowanie meta-cech za pomocą walidacji krzyżowej (**StratifiedKFold** z 5 podziałami):
 - Uzyskuje się probabilistyczne predykcje dla klasy pozytywnej z każdego modelu bazowego na **X_train_scaled** za pomocą **cross_val_predict**.
 - Tworzy się macierz meta-cech **meta_features_train** z predykcji **Random Forest**, **XGBoost** i **Gradient Boosting**.
4. Trenowanie meta-modelu:
 - Inicjalizuje się **LogisticRegression** z **solver='liblinear'** i **random_state=42**.
 - Przeprowadza się trening meta-modelu na **meta_features_train** i **y_train**.
5. Generowanie predykcji na zbiorze testowym:
 - Przeprowadza się trening każdego modelu bazowego na całym **X_train_scaled**.
 - Uzyskuje się probabilistyczne predykcje dla klasy pozytywnej z każdego wytrenowanego modelu bazowego na **X_test_scaled**.

- Tworzy się macierz meta-cech **meta_features_test** z predykcji **Random Forest**, **XGBoost** i **Gradient Boosting** dla zbioru testowego.
6. Zwracanie wyników: Wracany jest wytrenowany **meta_model**, **meta_features_test** i **y_test**.

3.3.5. Kod

```
# 1. Skalowanie danych
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 2. Inicjalizacja i optymalizacja modeli bazowych
rf_clf = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    random_state=42
)

xgb_clf = xgb.XGBClassifier(
    n_estimators=200,
    use_label_encoder=False,
    eval_metric='logloss',
    learning_rate=0.05,
    max_depth=5,
    random_state=42
)

gb_clf = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42
)

# 3. Walidacja krzyzowa do generowania meta-cech
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

rf_preds_train = cross_val_predict(
    rf_clf,
```



```

X_train_scaled,
y_train,
cv=skf,
method='predict_proba'
)[: , 1]

xgb_preds_train = cross_val_predict(
xgb_clf,
X_train_scaled,
y_train,
cv=skf,
method='predict_proba'
)[: , 1]

gb_preds_train = cross_val_predict(
gb_clf,
X_train_scaled,
y_train,
cv=skf,
method='predict_proba'
)[: , 1]

meta_features_train = pd.DataFrame({
'rf_preds': rf_preds_train,
'xgb_preds': xgb_preds_train,
'gb_preds': gb_preds_train
})

# 4. Trening meta-modelu
meta_model = LogisticRegression(
solver='liblinear', random_state=42
)
meta_model.fit(meta_features_train, y_train)

# 5. Predykcje na zbiorze testowym
rf_clf.fit(X_train_scaled, y_train)
xgb_clf.fit(X_train_scaled, y_train)
gb_clf.fit(X_train_scaled, y_train)

```

```

rf_preds_test = rf_clf.predict_proba(X_test_scaled)[: , 1]
xgb_preds_test = xgb_clf.predict_proba(X_test_scaled)[: , 1]
gb_preds_test = gb_clf.predict_proba(X_test_scaled)[: , 1]

meta_features_test = pd.DataFrame({
    'rf_preds': rf_preds_test,
    'xgb_preds': xgb_preds_test,
    'gb_preds': gb_preds_test
})

return meta_model, meta_features_test, y_test

```

3.3.6. Właściwości analityczne

1. Efektywność:

- Wykorzystanie różnorodnych algorytmów predykcyjnych, w tym opartych na zespołach predyktorów i technikach wzmacniania gradientowego, pozwala na uchwycenie złożonych zależności w danych.
- Zastosowanie walidacji krzyżowej do generowania meta-cech minimalizuje ryzyko przeuczenia.
- Meta-model (**regresja logistyczna**) efektywnie łączy predykcje różnych modeli bazowych.

2. Stabilność:

- Modele bazowe (Random Forest i Gradient Boosting) są stosunkowo stabilne dzięki swoim mechanizmom agregacji i korekcji błędów.
- Stacking z walidacją krzyżową pomaga w uzyskaniu bardziej stabilnych predykcji meta-modelu.

3. Złożoność obliczeniowa:

- Trening wielu modeli bazowych i meta-modelu zwiększa złożoność obliczeniową w porównaniu do pojedynczych modeli.
- Walidacja krzyżowa dodatkowo wydłuża czas treningu.

4. Wydajność: Łączenie mocnych stron różnych algorytmów często prowadzi do wyższej dokładności predykcji niż pojedyncze modele.

3.4. CZWARTA METODA (METODA20)

3.4.1. Pomysł na metodę

Metoda ta powstała na bazie idei integracji wielu modeli, podobnie jak w **Metoda17** i **Metoda18**, gdzie różne klasyfikatory były łączone w celu poprawy ogólnej wydajności pre-

dykcyjnej. Inspiracją były również metody stacking, takie jak **Metoda18**, gdzie predykcje modeli bazowych służyły jako cechy dla meta-modelu.

W **Metoda20** połączono LightGBM, CatBoost, Random Forest i Gradient Boosting. LightGBM i CatBoost to zaawansowane algorytmy boostingowe znane ze swojej szybkości i efektywności. Random Forest jest modelem opartym na zespołach losowo tworzonych predyktorów, a Gradient Boosting sekwencyjnie buduje modele korygujące błędy poprzedników. Podobnie jak w **Metoda18**, regresja logistyczna służy tutaj jako meta-model, integrując probabilistyczne predykcje tych czterech różnorodnych modeli bazowych. Celem jest wykorzystanie unikalnych mocnych stron każdego z tych algorytmów, aby osiągnąć wyższą dokładność i bardziej stabilne predykcje.

3.4.2. Wejściowe dane

Metoda `train_run20` przyjmuje następujące parametry wejściowe:

1. **X_train**: Macierz danych treningowych dla n próbek, m cech. Każdy wiersz reprezentuje próbkę, a każda kolumna cechę. Wartości w macierzy są zazwyczaj liczbowe.
2. **y_train**: Wektor etykiet dla danych treningowych. Rozmiar: n , gdzie n - liczba próbek w danych treningowych.
3. **X_test**: Macierz danych testowych, analogiczna do **X_train**, używana do oceny modelu.
4. **y_test**: Wektor etykiet testowych, odpowiadający danym testowym.

3.4.3. Wyjściowe dane

Metoda zwraca następujące elementy:

1. **meta_model**: Meta-model (regresja logistyczna) wytrenowany na predykcjach **LightGBM**, **CatBoost**, **Random Forest** i **Gradient Boosting**.
2. **meta_features_test**: Macierz meta-cech (predykcje modeli bazowych) dla danych testowych.
3. **y_test**: Wektor etykiet testowych.

3.4.4. Algorytm

1. Trenowanie modeli bazowych:
 - Przeprowadza się trening **LGBMClassifier** na **X_train** i **y_train**.
 - Przeprowadza się trening **CatBoostClassifier** na **X_train** i **y_train**.
 - Przeprowadza się trening **RandomForestClassifier** na **X_train** i **y_train**.
 - Przeprowadza się trening **GradientBoostingClassifier** na **X_train** i **y_train**.
2. Generowanie predykcji dla danych treningowych:
 - Uzyskuje się probabilistyczne predykcje klasy pozytywnej z każdego modelu bazowego na **X_train**.

- Tworzy się macierz meta-cech **meta_features_train**, gdzie każda kolumna zawiera predykcje jednego z modeli bazowych.
- 3. Trenowanie meta-modelu:
 - Przeprowadza się trening **LogisticRegression** na **meta_features_train** i **y_train**.
- 4. Generowanie predykcji dla danych testowych:
 - Uzyskuje się probabilistyczne predykcje klasy pozytywnej z każdego wytrenowanego modelu bazowego na **X_test**.
 - Tworzy się macierz meta-cech **meta_features_test**, analogicznie do treningowej.
- 5. Zwracanie wyników: Wracany jest wytrenowany **meta_model**, **meta_features_test** i **y_test**.

3.4.5. Kod

```

1 lgb_clf = LGBMClassifier(
2   n_estimators=100,
3   random_state=42
4 ) # LightGBM
5 cat_clf = CatBoostClassifier(
6   iterations=100,
7   verbose=0,
8   random_state=42
9 ) # CatBoost
10 rf_clf = RandomForestClassifier(
11   n_estimators=100,
12   random_state=42
13 ) # Random Forest
14 gb_clf = GradientBoostingClassifier(
15   n_estimators=100,
16   random_state=42
17 ) # Gradient Boosting
18
19 lgb_clf.fit(X_train, y_train)
20 cat_clf.fit(X_train, y_train)
21 rf_clf.fit(X_train, y_train)
22 gb_clf.fit(X_train, y_train)
23
24 lgb_preds_train = lgb_clf.predict_proba(X_train)[: , 1]
25 cat_preds_train = cat_clf.predict_proba(X_train)[: , 1]
26 rf_preds_train = rf_clf.predict_proba(X_train)[: , 1]
27 gb_preds_train = gb_clf.predict_proba(X_train)[: , 1]
28
29 meta_features_train = pd.DataFrame({
30   'lgb_preds': lgb_preds_train,
31   'cat_preds': cat_preds_train,
32   'rf_preds': rf_preds_train,

```

```

33 'gb_preds': gb_preds_train
34 })
35
36 meta_model = LogisticRegression()
37 meta_model.fit(meta_features_train, y_train)
38
39 lgb_preds_test = lgb_clf.predict_proba(X_test)[: , 1]
40 cat_preds_test = cat_clf.predict_proba(X_test)[: , 1]
41 rf_preds_test = rf_clf.predict_proba(X_test)[: , 1]
42 gb_preds_test = gb_clf.predict_proba(X_test)[: , 1]
43
44 meta_features_test = pd.DataFrame({
45 'lgb_preds': lgb_preds_test,
46 'cat_preds': cat_preds_test,
47 'rf_preds': rf_preds_test,
48 'gb_preds': gb_preds_test
49 })
50 return meta_model, meta_features_test, y_test

```

Listing 3.1: Kod do 20 metody

3.4.6. Właściwości analityczne

1. **Efektywność:** Łączenie predykcji wielu modeli (boostingowych i opartych na zespołach predyktorów) ma na celu zwiększenie dokładności i robustności predykcji.
2. **Wykorzystanie różnorodności modeli:** Zastosowanie różnych typów algorytmów pozwala na uwzględnienie różnych aspektów danych i wzorców.
3. **Meta-model:** Regresja logistyczna skutecznie integruje predykcje modeli bazowych, ucząc się, jak najlepiej je łączyć.
4. **Złożoność obliczeniowa:** Trening wielu złożonych modeli bazowych może być kosztowny obliczeniowo, ale predykcja jest stosunkowo szybka.
5. **Wydajność:** Oczekuje się, że kombinacja tych technik modelowania zapewni wysoką wydajność w zadaniu klasyfikacji.

4. SZCZEGÓŁY IMPLEMENTACYJNE

4.1. TECHNOLOGIE

W tej sekcji omówiono technologie wykorzystane w projekcie.

Aplikacja została napisana w języku programowania Python. Technologia ta została wybrana ze względu na szeroki wachlarz dostępnych bibliotek, frameworków oraz narzędzi. Doskonale nadaje się do tworzenia aplikacji wykorzystujących techniki uczenia maszynowego. Zainstalowana wersja to **Python 3.12.6**. Implementacja korzysta z następujących bibliotek:

Aby zmienić wszystkie wystąpienia na **w** podanym tekście, wystarczy zamienić każde na **.** Oto poprawiony tekst:

- **tensorflow** to wszechstronna biblioteka używana do budowy modeli uczenia maszynowego i głębokiego uczenia. Oferuje szerokie wsparcie dla sieci neuronowych, takich jak CNN i RNN. W projekcie została wykorzystana do implementacji zaawansowanych modeli uczenia maszynowego. **Użyta wersja: '2.18.0'**.
- **keras** jest wysokopoziomowym API opartym na TensorFlow, ułatwiającym budowę i trenowanie modeli głębokiego uczenia. W projekcie została wykorzystana do definiowania warstw, architektury sieci i przygotowania modeli. **Użyta wersja: '3.6.0'**.
- **scikit-learn** zawiera szeroką gamę narzędzi do uczenia maszynowego, w tym modele klasyfikacji, regresji, klasteryzacji, a także metody wstępnego przetwarzania danych. W projekcie była używana do transformacji danych i budowy modeli klasyfikacyjnych. **Użyta wersja: '1.5.2'**.
- **xgboost** to wydajna biblioteka do gradientowego zwiększania liczby drzew, szczególnie skuteczna w zadaniach klasyfikacyjnych i regresyjnych. W projekcie została wykorzystana do tworzenia modeli zespołowych. **Użyta wersja: '2.1.2'**.
- **pandas** to biblioteka przeznaczona do manipulacji danymi i analizy. Ułatwia obsługę dużych zbiorów danych w formie tabelarycznej. W projekcie używana była do wczytywania, czyszczenia i przekształcania danych. **Użyta wersja: '2.2.3'**.
- **numpy** to podstawowa biblioteka do obliczeń numerycznych w Pythonie. Dostarcza funkcje do operacji na wielowymiarowych tablicach, generowania liczb losowych oraz wykonywania obliczeń matematycznych. **Użyta wersja: '1.26.4'**.
- **matplotlib** to biblioteka służąca do wizualizacji danych w postaci wykresów i diagramów. W projekcie była używana do przedstawienia wyników w postaci graficznej. **Użyta wersja: '3.9.2'**.

- **seaborn** to rozszerzenie dla **matplotlib**, które umożliwia tworzenie bardziej atrakcyjnych i zaawansowanych wizualizacji statystycznych. W projekcie była używana do generowania szczegółowych wykresów danych. **Użyta wersja: '0.13.2'**.
- **transformers** to biblioteka rozwijana przez Hugging Face, używana do przetwarzania języka naturalnego (NLP). Zawiera zaimplementowane modele, takie jak BERT, GPT, i RoBERTa. W projekcie wykorzystano ją do klasyfikacji tekstu. **Użyta wersja: '4.46.2'**.
- **torch** (PyTorch) to elastyczna biblioteka do głębokiego uczenia, wspierająca dynamiczne definiowanie sieci neuronowych. W projekcie była używana do tworzenia modeli opartych na sieciach neuronowych i obliczeń tensorowych. **Użyta wersja: '2.5.1'**.

4.2. WYMAGANIA SYSTEMOWE

4.2.1. Sprzęt

W poniższej podsekcji wymieniono minimalne wymagania sprzętowe do uruchomienia aplikacji i testów.

- Procesor: co najmniej 4 rdzenie, 8 wątków, częstotliwość bazowa 3,60 GHz i 6 MB pamięci cache. W testach wykorzystano procesor Intel® Core™ i7 10700f.
- Pamięć: co najmniej 16 GB RAM.
- Karta graficzna: dedykowany układ graficzny z co najmniej 4 GB pamięci. W testach wykorzystano AMD® Radeon RX 6400 (4 GB).
- Dysk: 4,5 GB wolnej przestrzeni na dysku.

4.2.2. Oprogramowanie

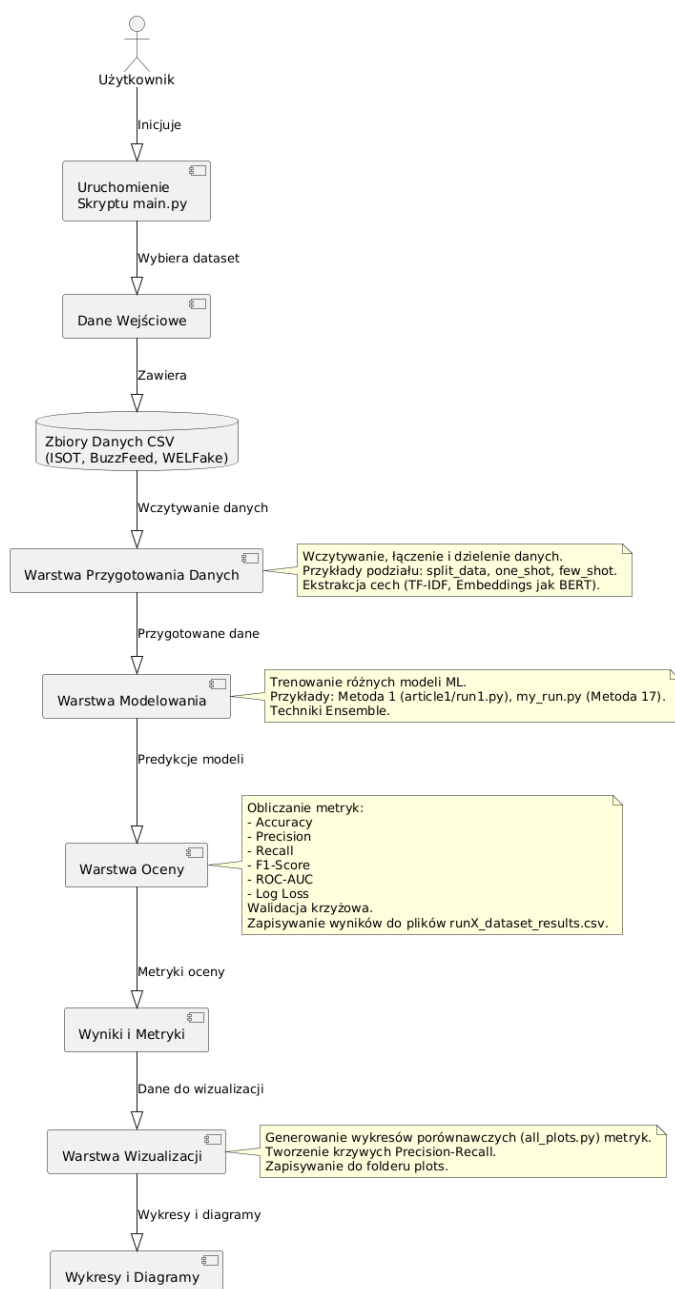
Poniżej wymieniono minimalne wymagania dotyczące oprogramowania, niezbędne do uruchomienia aplikacji i testów.

- Zainstalowany Python (wersja określona w Sekcji 4.1).
- Zainstalowana najnowsza wersja pip.
- Instalacja bibliotek i frameworków odbywa się za pomocą pliku `requirements.txt`. Aby zainstalować wszystkie wymagane zależności, należy w terminalu uruchomić polecenie `pip install -r requirements.txt`.
- Umieszczenie danych testowych w odpowiednim podfolderze projektu: `projekt\datasets\nazwa`. Są dostępne tylko podane trzy opcje datasetów: **WELFake_dataset**, **ISOT_dataset** oraz **BuzzFeed_dataset**. Dane testowe powinny być zapisane w plikach CSV o następującej strukturze:
 - Kolumny: `title`, `text`, `subject`, `date`.
- W folderze każdego datasetu powinny znajdować się dwa pliki danych:
 - `Fake.csv` - zawierający fałszywe wiadomości.

- `True.csv` - zawierający prawdziwe wiadomości.
- Wystarczająca ilość miejsca na dane.

4.3. ARCHITEKTURA SYSTEMU

System został zaprojektowany do analizy i klasyfikacji tekstów wiadomości jako fałszywe lub prawdziwe, wykorzystując różnorodne algorytmy uczenia maszynowego oraz techniki ensemble. Składa się z czterech głównych warstw, które odpowiadają za przetwarzanie danych, trenowanie modeli, ocenę ich skuteczności oraz wizualizację wyników. Poniżej przedstawiono szczegóły każdej warstwy. Całość jest pokazana na rysunku 4.1.



Rys. 4.1: Diagram architektury systemu.

4.3.1. Warstwa Przygotowania Danych

Dane wejściowe są wczytywane z plików CSV: `Fake.csv`, oznaczonych etykietą 0 oraz `True.csv`, oznaczonych etykietą 1, przechowywanych w podfolderach datasetów: `ISOT_dataset`, `BuzzFeed_dataset`, `WELFake_dataset`. Moduł `common.py` zawiera funkcje takie jak `load_and_preprocess_data`, które integrują dane i przygotowują je do dalszej obróbki. Podział na zbiory treningowe i testowe realizowany jest za pomocą funkcji `split_data`, `split_data_one_shot` oraz `split_data_few_shot`, umożliwiając różne strategie podziału danych. Ekstrakcja cech obejmuje:

- **TF-IDF**: przekształca tekst na reprezentacje numeryczne za pomocą wektorów ważonych częstotliwością terminów.
- **BERT/RoBERTa/Sentence Transformers**: generuje zaawansowane osadzenia kontekstowe tekstu.

4.3.2. Warstwa Trenowania Modeli

Proces uruchamiany jest poprzez skrypt `main.py` w konsoli. Użytkownik wybiera reprezentację kontekstową: 1 - BERT, 2 - RoBERTa, 3 - Sentence Transformers; lub tryb "all" dla wszystkich opcji. Następnie określa numer metody: od 1 do 20; lub zakres metod, tryb podziału danych: klasyczny, one-shot, few-shot; oraz dataset: ISOT, BuzzFeed, WELFake. Każda implementacja standardowa, od `metoda1` do `metoda16`, znajduje się w osobnym folderze `articleX`, gdzie X to numer metody, co pozwala na ich niezależne rozwijanie.

Autorskie metody (`metoda17` do `metoda20`) są zaimplementowane w pliku `my_run.py` i dostępne bezpośrednio w głównym katalogu projektu. Modele obejmują:

- Algorytmy uczenia maszynowego, takie jak Gradient Boosting, Random Forest, LightGBM czy CatBoost.
- Sieci neuronowe oparte na TensorFlow z dynamicznym ważeniem próbek.
- Połączenia predykcji za pomocą meta-modeli, regresji logistycznej.

Przykładowe uruchomienie pokazano na rysunku 4.2.

```

Wybierz reprezentację kontekstową (1-3) lub wpisz 'all', aby uruchomić wszystkie:
1 - BERT
2 - RoBERTa
3 - Sentence Transformers
3
Wybierz który dataset zostanie użyty:
1 - ISOT
2 - BuzzFeed
3 - WELFake
2
Wybierz numer metody początkującej (1-20) lub wpisz 'all', aby uruchomić wszystkie:
18
Wybierz numer metody ostatecznej (1-20)
18
Wybierz tryb podziału danych (1-3) lub wpisz 'all', aby uruchomić wszystkie:
1 - Klasyczny split
2 - One Shot
3 - Few Shot
3

```

Rys. 4.2: Przykładowe uruchomienie aplikacji.

4.3.3. Warstwa Oceny Wyników

Ocena skuteczności modeli przeprowadzana jest za pomocą funkcji `evaluate_model` z modułu `common.py`. Funkcja ta oblicza kluczowe metryki, w tym:

- Dokładność (Accuracy),
- Precyzję (Precision),
- Recall,
- F1-Score,
- ROC-AUC,
- MCC,
- Log Loss,
- Cohen's Kappa.

Dodatkowo obliczane są metryki walidacji krzyżowej (CV Accuracy). Wyniki zapisywane są w osobnych plikach `runX_dataset_results.csv` i `runX_dataset_pr_curve.csv` w folderach `results_X_Y`, gdzie X określa typ reprezentacji, a Y tryb podziału danych, przykładowo `results_3_few_shot`. Każdy folder przechowuje wyniki specyficzne dla danego eksperymentu.

4.3.4. Warstwa Wizualizacji

Analiza graficzna wyników realizowana jest przez skrypt `all_plots.py`, który przetwarza dane z plików CSV zapisanych w folderach `results_X_Y`. Generowane są:

- Wykresy słupkowe porównujące metryki, takie jak Dokładność, Czas wykonania czy Log Loss oraz inne, dla różnych metod i zbiorów danych (BuzzFeed, ISOT, WELFake).
- Krzywe Precision-Recall dla każdej metody i zbioru danych.

Wykresy są zapisywane w folderze `plots`, zorganizowanym w podfoldery według datasetów i typów eksperymentów, co ułatwia ich przeglądanie.

BIBLIOGRAFIA

- [1] Adedoyin S. Adeyemi, Sunday O. Asakpa, Kehinde H. Bello, Olajide N. Saliu, Olabisi M. Olaoye, and Nike T. Toye. Fake news detection using ensemble methods: an empirical evaluation of bagging and boosting algorithms. *International Journal of Advances in Engineering and Management (IJAEM)*, 6(06):317–325, 2024. Uzyskano dostęp: 23 Stycznia 2025.
- [2] Iftikhar Ahmad, Muhammad Yousaf, Suhail Yousaf, and Muhammad Ovais Ahmad. Fake news detection using machine learning ensemble methods. *Complexity*, 2020:1–11, 10 2020. Uzyskano dostęp: 23 Stycznia 2025.
- [3] Abdullah Marish Ali, Fuad A. Ghaleb, Bander Ali Saleh Al-Rimy, Fawaz Jaber Alsolami, and Asif Irshad Khan. Deep ensemble fake news detection model using sequential deep learning technique. *Sensors*, 22(18), 2022. Uzyskano dostęp: 23 Stycznia 2025.
- [4] Esma Aïmeur, Sabrine Amri, and Gilles Brassard. Fake news, disinformation and misinformation in social media: a review. *Social Network Analysis and Mining*, 13(1):30, 2023. Uzyskano dostęp: 23 Stycznia 2025.
- [5] M.A. Ganaie, Minghui Hu, A.K. Malik, M. Tanveer, and P.N. Suganthan. Ensemble deep learning: A review. *Engineering Applications of Artificial Intelligence*, 115:105151, 2022. Uzyskano dostęp: 23 Stycznia 2025.
- [6] Alireza Ghorbanali, Mohammad Karim Sohrabi, and Farzin Yaghmaee. Ensemble transfer learning-based multimodal sentiment analysis using weighted convolutional neural networks. *Information Processing & Management*, 59(3):102929, 2022. Uzyskano dostęp: 23 Stycznia 2025.
- [7] Saqib Hakak, Mamoun Alazab, Suleman Khan, Thippa Reddy Gadekallu, Praveen Kumar Reddy Maddikunta, and Wazir Zada Khan. An ensemble machine learning approach through effective feature extraction to classify fake news. *Future Generation Computer Systems*, 117:47–58, 2021. Uzyskano dostęp: 23 Stycznia 2025.
- [8] Azal Ahmad Khan, Omkar Chaudhari, and Rohitash Chandra. A review of ensemble learning and data augmentation models for class imbalanced problems: Combination, implementation and evaluation. *Expert Systems with Applications*, 244:122778, 2024. Uzyskano dostęp: 23 Stycznia 2025.
- [9] Junaed Younus Khan, Md. Tawkat Islam Khondaker, Sadia Afroz, Gias Uddin, and Anindya Iqbal. A benchmark study of machine learning models for online fake news detection. *Machine Learning with Applications*, 4:100032, 2021. Uzyskano dostęp: 23 Stycznia 2025.
- [10] Atik Mahabub. A robust technique of fake news detection using ensemble voting classifier and comparison with other classifiers. *SN Applied Sciences*, 2(4):525, 2020. Uzyskano dostęp: 23 Stycznia 2025.
- [11] Shubha Mishra, Piyush Shukla, and Ratish Agarwal. Analyzing machine learning enabled

- fake news detection techniques for diversified datasets. *Wireless Communications and Mobile Computing*, 2022(1):1575365, 2022. Uzyskano dostęp: 23 Kwietnia 2025.
- [12] Parth Patel, Shubham Patel, and Yashkumar Patel. Ensemble based approach for fake news detection. *International Research Journal of Engineering and Technology (IRJET)*, 08(05):2732, 2021. Uzyskano dostęp: 23 Stycznia 2025.
 - [13] Shafin Rahman, Salman Khan, and Fatih Porikli. A unified approach for conventional zero-shot, generalized zero-shot, and few-shot learning. *IEEE Transactions on Image Processing*, PP, 06 2017. Uzyskano dostęp: 23 Marca 2025.
 - [14] Amit Neil Ramkissoo and Wayne Goodridge. Enhancing the predictive performance of credibility-based fake news detection using ensemble learning. *The Review of Socionetwork Strategies*, 16(2):259–289, 2022. Uzyskano dostęp: 23 Stycznia 2025.
 - [15] Harita Reddy, Namratha Raj, Manali Gala, and Annappa Basava. Text-mining-based fake news detection using ensemble methods. *Machine Intelligence Research*, 17(2):210–221, 2020. Uzyskano dostęp: 23 Stycznia 2025.
 - [16] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *CoRR*, abs/1908.10084, 2019. Uzyskano dostęp: 23 Marca 2025.
 - [17] Arjun Roy, Kingshuk Basak, Asif Ekbal, and Pushpak Bhattacharyya. A deep ensemble framework for fake news detection and classification, 2018. Uzyskano dostęp: 23 Stycznia 2025.
 - [18] Giovanni Santia and Jake Williams. Buzzface: A news veracity dataset with facebook user commentary and egos. *Proceedings of the International AAAI Conference on Web and Social Media*, 12(1):531–540, Jun. 2018. Uzyskano dostęp: 23 Kwietnia 2025.
 - [19] K V, B B Al-onazi, V Simic, E B Tirkolae, and C Jana. Deepfnd: An ensemble-based deep learning approach for the optimization and improvement of fake news detection in digital platform. *PeerJ Computer Science*, 9:e1666, 2023. Uzyskano dostęp: 23 Stycznia 2025.
 - [20] Pawan Verma, Prateek Agrawal, Ivone Amorim, and Radu Prodan. Welfake: Word embedding over linguistic features for fake news detection. *IEEE Transactions on Computational Social Systems*, PP:1–13, 04 2021. Uzyskano dostęp: 23 Stycznia 2025.
 - [21] Pawan Kumar Verma, Prateek Agrawal, Ivone Amorim, and Radu Prodan. Welfake: Word embedding over linguistic features for fake news detection. *IEEE Transactions on Computational Social Systems*, 8(4):881–893, 2021. Uzyskano dostęp: 23 Kwietnia 2025.

SPIS RYSUNKÓW

2.1	Schemat metody 15.	21
2.2	Schemat metody 2.	23
2.3	Schemat metody 3.	24
2.4	Schemat metody 5.	26
2.5	Porównanie Accuracy dla ISOT Fake News Dataset dla różnych uruchomień metod. .	42
2.6	Porównanie Precision dla ISOT dla różnych uruchomień metod.	43
2.7	Porównanie Recall dla ISOT dla różnych uruchomień metod.	44
2.8	Porównanie F1-Score dla ISOT dla różnych uruchomień metod.	44
2.9	Porównanie Log-Loss dla ISOT dla różnych uruchomień metod.	45
2.10	Porównanie MCC dla ISOT dla różnych uruchomień metod.	46
2.11	Porównanie ROC-AUC dla ISOT dla różnych uruchomień metod.	46
2.12	Porównanie Cohen's Kappa dla ISOT dla różnych uruchomień metod.	47
2.13	Porównanie Mean dla ISOT dla różnych uruchomień metod.	48
2.14	Porównanie Execution Time dla ISOT dla różnych uruchomień metod.	49
4.1	Diagram architektury systemu.	67
4.2	Przykładowe uruchomienie aplikacji.	69

Dodatki